Algorithmica (1992) 7: 137–177

Algorithmica
© 1992 Springer-Verlag New York Inc.

How To Find Steiner Minimal Trees in Euclidean d-Space

Warren D. Smith[1]

Abstract. This paper has two purposes. The first is to present a new way to find a Steiner minimum tree (SMT) connecting N sites in d-space, $d \geq 2$. We present (in Appendix 1) a computer code for this purpose. This is the only procedure known to the author for finding Steiner minimal trees in d-space for $d > 2$, and also the first one which fits naturally into the framework of "backtracking" and "branch-and-bound." Finding SMTs of up to $N = 12$ general sites in d-space (for any d) now appears feasible.

We tabulate Steiner minimal trees for many point sets, including the vertices of most of the regular and Archimedean d-polytopes with ≤ 16 vertices. As a consequence of these tables, the Gilbert–Pollak conjecture is shown to be false in dimensions 3–9. (The conjecture remains open in other dimensions; it is probably false in all dimensions d with $d \geq 3$, but it is probably true when $d = 2$.)

The second purpose is to present some new theoretical results regarding the asymptotic computational complexity of finding SMTs to precision ε.

We show that in two-dimensions, Steiner minimum trees may be found exactly in exponential time $\mathbf{O}(C^N)$ on a real RAM. (All previous provable time bounds were superexponential.) If the tree is only wanted to precision ε, then there is an $(N/\varepsilon)^{\mathbf{O}(\sqrt{N})}$-time algorithm, which is *subexponential* if $1/\varepsilon$ grows only polynomially with N. Also, the *rectilinear* Steiner minimal tree of N points in the plane may be found in $N^{\mathbf{O}(\sqrt{N})}$ time.

J. S. Provan devised an $\mathbf{O}(N^6/\varepsilon^4)$-time algorithm for finding the SMT of a convex N-point set in the plane. (Also the rectilinear SMT of such a set may be found in $\mathbf{O}(N^6)$ time.) One therefore suspects that this problem may be solved exactly in polynomial time. We show that this suspicion is in fact true—if a certain conjecture about the size of "Steiner sensitivity diagrams" is correct.

All of these algorithms are for a "real RAM" model of computation allowing infinite precision arithmetic. They make no probabilistic or other assumptions about the input; the time bounds are valid in the worst case; and all our algorithms may be implemented with a polynomial amount of space. Only algorithms yielding the *exact* optimum SMT, or trees with lengths $\leq (1 + \varepsilon) \times$ optimum, where ε is arbitrarily small, are considered here.

Key Words. Steiner trees, Gilbert–Pollak conjecture, Subexponential algorithms, Regular polytopes, Sensitivity diagrams.

0. Basic Facts and Definitions. This section presents basic facts and definitions which we will need. Readers familiar with Steiner tree terminology may skip to Section 1.

A set of real numbers is said to be in "general position" if they satisfy no nontrivial algebraic equation with integer coefficients. A positive real-valued function $f(N)$, which may be bounded for all sufficiently large N by

$$(C_1)^N < f(N) < (C_2)^N,$$

[1] AT&T Bell Laboratories, Murray Hill, NJ 07974, USA.

Received November, 1988; revised September, 1989, and February, 1990. Communicated by F. K. Hwang.

where $1 < C_1 \leq C_2$, is said to be "exponential." A function which grows faster than any upper bound of this type is "superexponential," and one which cannot be bounded by any lower bound of this type is "subexponential." **Z** and **R** represent the sets of integers and real numbers, respectively.

The Steiner minimal tree (SMT) on N particles $\vec{x}_1 \cdots \vec{x}_N$ in d-space, $d \geq 2$, is the shortest tree containing the sites. A "Steiner tree" is a tree of line segments with the following properties:

1. It contains N "regular sites" $\vec{x}_1 \cdots \vec{x}_N$, and possibly K additional "Steiner nodes" $\vec{x}_{N+1} \cdots \vec{x}_{N+K}$.
2. Each Steiner node has valence 3, and the edges emanating from it lie in a plane and have mutual angle 120°.
3. Each regular site has valence between 1 and 3 (and generally ≤ 2).
4. $0 \leq K \leq N - 2$.

It is easily proved that an SMT must be a Steiner tree and no angle of SMT is $< 120°$. Any three rays intersecting at mutual 120° angles must lie in a plane, hence Property 2 follows. Property 4 is a consequence of Properties 2 and 3.

The shortest tree containing all the regular sites, but which is *not allowed* to have Steiner points, is called the minimum spanning tree (MST). Even in the Euclidean plane, determining SMT is NP-hard [15], but MST may be easily found by a number of simple polynomial-time algorithms [32], [40], [41].

A "full Steiner tree" (FST) is a Steiner tree with all original sites having valence 1, or equivalently, a Steiner tree is "full" iff it has exactly $N - 2$ Steiner nodes. It is easily shown that any Steiner tree is a union of edge-disjoint full Steiner trees. (In particular, any edge of SMT joining two original sites is also an edge of MST.)

The "Steiner point" S of three points, A, B, C, may be calculated as follows. Let the triangle $\triangle ABC$ have angles θ_A, θ_B, θ_C. If any angle $\theta_i \geq 120°$, then the Steiner point is i, and SMT = MST = the two shortest sides of $\triangle ABC$. Otherwise, erect equilateral triangles ("Napoleon triangles") on the sides of $\triangle ABC$ with far vertices V_A, V_B, V_C. Then S lies at the common intersection of line segments iV_i, which is also the common intersection of the circumcircles of the three equilateral triangles, and also

$$\overline{AV_A} = \overline{BV_B} = \overline{CV_C} = \overline{AS} + \overline{BS} + \overline{CS}. \tag{1}$$

If the triangle has sides a, b, c and area Q, then the length of the SMT is L where

$$2L^2 = a^2 + b^2 + c^2 + 4\sqrt{3}Q. \tag{2}$$

The length of the branch $\overline{AS}$ of the SMT is

$$\frac{L}{3} + \frac{b^2 + c^2 - 2a^2}{3L} = \frac{4\sqrt{3}Q + 3b^2 + 3c^2 - 3a^2}{6L} \tag{3}$$

and the first barycentric coordinate of S is proportional to $1/\overline{AS}$. (The remaining branches and barycentric coordinates are given by the obvious symmetric formulas.)

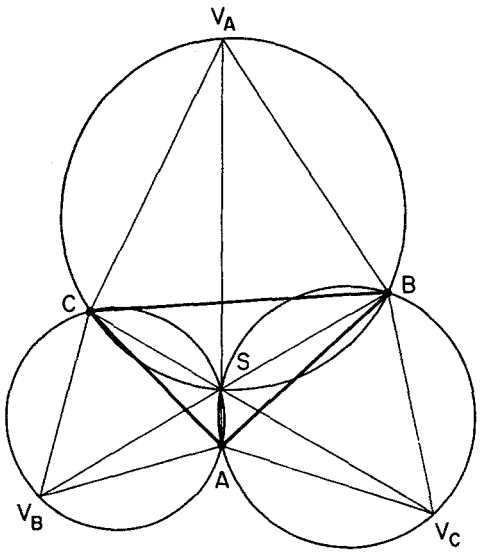


Fig. 1. "Napoleon's" construction of Steiner point S for a triangle ΔABC.

The combinatorial structure (i.e., the adjacencies) of a tree is called (by the vast weight of previous usage, even though it is not a particularly good word) its "topology." A tree which corresponds to some topology, but with certain edges shrunk to zero length, is said to be "degenerate." All nonfull topologies are thus degenerate full topologies. For further Steiner tree terminology, see the papers by Gilbert and Pollak [18] and Hwang and Richards [26]; for general tree terminology, see [49].

1. Introduction. After Gilbert and Pollak [18], all practical algorithms for finding the Steiner minimal tree of N point sites in d-space (although, in fact, I know of no previous algorithmic study for $d \geq 3$) followed essentially the same master plan. This plan is:

GP1. Find all Steiner tree "topologies" on N regular sites and K, $0 \leq K \leq N-2$, Steiner points. (Gilbert and Pollak showed that there were exactly

$$\binom{N}{K+2}\frac{(N+K-2)!}{K!\,2^K}$$

Steiner tree topologies with N regular sites and K Steiner points.)

GP2. For each topology, optimize the coordinates of the Steiner points in order to find the shortest possible Euclidean embedding of that topology.

GP3. Output the shortest tree found in this way.

By summing over K and doing an asymptotic analysis, it may be shown that the run-time of the Gilbert–Pollak algorithm is $\mathbf{O}(3.729^N N!)$ steps, where a "step"

consists of generating a new tree topology and optimizing its Steiner point coordinates.

All the SMT computer programs which I know of have followed some version of this plan. It is worth pointing out a few reasons why the d-dimensional SMT problem differs dramatically from the two-dimensional problem.

First, in two dimensions there is a simple algorithm (due to Hwang and Melzak, and explained in Section 3.4) for finding the exact minimum length tree with some prespecified (nondegenerate) topology, in $\mathbf{O}(N)$ steps, each step involving a rational operation or a square root. The fact that such a simple algorithm exists is ultimately due to the fact that the orientation of any edge in a two-dimensional full Steiner tree is completely determined by the orientation of any one of its edges (since all Steiner angles are 120°) (Hwang and Weng [28]). This is a property which disappears in higher dimensions!

I believe that no such algorithm can exist in any dimension $d \geq 3$. This claim is made more precise in the following theorems and conjecture; Theorem 1 shows why the situation in two dimensions is simple, while Theorem 2 and Conjecture 1 show why it is difficult in higher dimensions.

THEOREM 1. *The coordinates of the Steiner points in the SMT of N regular sites in the plane are rational functions of the coordinates of the regular sites, with coefficients in the field* $Q(\sqrt{3})$. *The square of the total length of a full SMT is a quadratic function of the regular site coordinates, with coefficients in* $Q(\sqrt{3})$. *Also, all these points and lengths may be constructed from the regular sites using a straightedge and compass. In fact, if all the regular sites have coordinates in* $\mathbf{Z}^2$, *then twice the square of the total length of the SMT is a number of the form* $A + B\sqrt{3}$, $A, B \in \mathbf{Z}$.

PROOF SKETCH. The proof is simply an analysis of the Hwang–Melzak algorithm of Section 3.4, and also using the observation that all of the reduced trees produced by this algorithm have the same length as the original. □

Already in 1968, Gilbert and Pollak knew the situation in higher dimensions was more complicated since the (full) SMT for the vertices of the regular 4-simplex (with coordinates being the rows of the identity matrix I_5) has length $\frac{1}{2}\sqrt{5} + \frac{1}{2}\sqrt{3} + \sqrt{6}$. In fact,

THEOREM 2. *Given four regular sites as general points in 3-space such that their SMT is a full topology (any small perturbation of a regular tetrahedron will do), the coordinates of the two Steiner points may be expressed as the root of an essentially general (e.g., generally irreducible) eighth-degree polynomial. Therefore, these points are not obtainable by any straightedge and compass construction, nor by solving any finite sequence of equations of degree* ≤ 7.

PROOF SKETCH. The proof here proceeds in two stages. The first stage is to realize [18], [36] that the length of the SMT is the same as the length of the longest line

segment joining two "Melzak circles." (The Melzak circle of two point sites in $\mathbf{R}^3$ is the locus of the points which form an equilateral triangle with the two given sites.)

The second stage is to analyze the algebraic complexity of the circle-distance problem. This analysis is extremely complicated, so we only sketch the details. Let the two circles have centers $\vec{c}_i$, and radius *vectors* $\vec{R}_i$, $i = 1, 2$. (The "radius vector" of a circle in 3-space is perpendicular to the plane of the circle, and has length equal to the circle's radius.) Let the endpoints of the desired line segment be $\vec{x}_1$ and $\vec{x}_2$. Then the six equations that we want to solve for the six unknowns $\vec{x}_i$, are that the following 6 quantities (actually three quantities but with $i = 1, 2$ for each) are zero.

$$(\mathrm{Q1}_i) \qquad (\vec{x}_i - \vec{c}_i)\cdot\vec{R}_i = 0,$$

$$(\mathrm{Q2}_i) \qquad |\vec{x}_i - \vec{c}_i|^2 - |\vec{R}_i|^2 = 0,$$

$$(\mathrm{Q3}_i) \qquad (\vec{x}_1 - \vec{x}_2)\cdot[\vec{R}_i \times (\vec{x}_i - \vec{c}_i)] = 0.$$

By using an appropriately rotated coordinate system centered on $\vec{c}_1$ and with Z-axis in the direction of $\vec{R}_1$, we may set $\vec{x}_1 = (\mathrm{x1}, \mathrm{y1}, 0)^{\mathrm{T}}$, $\vec{x}_2 = (\mathrm{x2}, \mathrm{y2}, \mathrm{z2})^{\mathrm{T}}$, $\vec{c}_1 = (0, 0, 0)^{\mathrm{T}}$, $\vec{c}_2 = (0, cy, cz)^{\mathrm{T}}$, $\vec{R}_1 = (0, 0, 1)^{\mathrm{T}}$, $\vec{R}_2 = (Rx, Ry, Rz)^{\mathrm{T}}$. I now undertook the following computation:

Realize $(\mathrm{Q1}_1)$ is the trivial equation $0 = 0$, so forget it in what follows.
Use linear equation $(\mathrm{Q1}_2)$ to eliminate variable z2 in all other equations.
$T_1 = \mathrm{Resultant}_{\mathrm{y2}}((\mathrm{Q2}_2), (\mathrm{Q3}_1))$,
$T_2 = \mathrm{Resultant}_{\mathrm{y2}}((\mathrm{Q3}_1), (\mathrm{Q3}_2))$,
$T_3 = \mathrm{Resultant}_{\mathrm{x2}}((T_1), (T_2))$,
$T_4 = \mathrm{Resultant}_{\mathrm{y1}}((T_3), (\mathrm{Q2}_1))$.
Upon removing trivial common factors from T_4, one is left with P.

The method of "resultants" is described in van der Waerden [51] and Uspensky [50]. This computation consecutively eliminated variables z2, y2, x2, y1, and by assumption we have $\mathrm{z1} = 0$, so at last I was left with the univariate eighth degree, generally irreducible, polynomial equation $P(\mathrm{x1}) = 0$, whose full expansion is about five pages long. Of course, once x1 is determined by solving this equation, the values of the other unknowns are readily deduced by solving some quadratic and linear equations.

It is very difficult to proceed further at the highest possible level of generality, so at this point let us specialize the problem to the particular case $cy = 107$, $cz = 109$, $Rx = 7$, $Ry = 11$, $Rz = 13$, when the polynomial $P(X)$ becomes

$$\begin{aligned}
&3836681837300X^8 + 1327137188582296X^7 + 230103593072913096X^6 \\
&\quad + 1157014127942903132 0X^5 + 1899999441515695508 45X^4 \\
&\quad - 418494309943902220X^3 - 9166674830821952906X^2 \\
&\quad + 2510097903471060X + 108706728928491225
\end{aligned}$$

an eight degree polynomial irreducible over the rationals. The discriminant [50] of this polynomial is not a square, as may be shown by direct computation. Upon factoring this polynomial modulo various small primes, one discovers that it is irreducible modulo 13, while modulo 31 it has the factorization

$$15(X-1)(X^7+13X^6+5X^5+10X^4+X^3-4X^2-10X+10).$$

According to theorems in [51], these facts more than suffice to show that the Galois group of P is the full symmetric group S_8, which proves that none of its solutions may be expressed in terms of radicals, and indeed nor may they be obtained by any finite sequence of solutions of equations of degree ≤ 7. (And, of course, this not only holds for our special case, but is in fact generally true.) □

CONJECTURE 1. *There is a sequence S_N of N-point sets in $\mathbf{R}^3$, such that for any positive integer C there is a value N_C such if $N > N_C$, then the length of the SMT of set S_N cannot be found by solving any finite sequence of equations of degree C.*

As we have just seen, this conjecture holds when $C = 7$. In fact, it seems likely that, for virtually any N-point set of regular sites in 3-space with a full SMT topology, one cannot find the SMT unless one is capable of solving (at least) general Nth-degree polynomial equations. In other words, there cannot be any finite algorithm generalizing the Hwang–Melzak method to higher dimensions, and therefore one is forced to resort to infinite converging processes.

Second, in two dimensions it is usually the case that the SMT is a union of edge disjoint full Steiner trees, each of which is quite small (it is quite rare that any of the FSTs involves more than three Steiner points). But in higher dimensions, the SMT usually consists of a *single* full Steiner tree involving all $N-2$ Steiner points. The reasons for these (experimentally observed) behaviors are not fully understood, but they have a great impact on the difficulty of finding SMTs in typical cases. P. Winter [53] developed various complicated, inherently two-dimensional "pushing" procedures for finding a small superset of elighble FSTs. Also the concept of the "Steiner hull" [18, §8.2], [27], [43] (which is difficult to generalize to high dimensions) sometimes permits one to divide a problem into smaller problems. All this allows a vast speedup in two dimensions. The latest and most refined probram [53], [7] makes it possible to solve "typical" 17-city problems in the plane in about 30 seconds on an IBM-4331. But there is apparently no way to generalize the qualities which make the two-dimensional SMT-finders fast to higher dimensions.

Part I: A Practical Procedure for Finding Steiner Minimal Trees

2. A Simple and Practical Backtracking Algorithm To Find Steiner Minimal Trees in *d*-Space. We do not depart from the Gilbert–Pollak plan in this paper. However, we have new approaches for implementing both step GP1 (the tree generation step) and step GP2 (the numerical optimization step). We will discuss the tree generation and overall algorithm in this section, and postpone discussion of the numerical optimization algorithms until the next section.

First, realize that we need only consider "full" Steiner topologies with $K = N - 2$. In the event that the optimum Steiner tree has $K < N - 2$, we will regard this merely as a "degenerate" full Steiner topology in which some of the Steiner points have merged. (Some previous algorithms could not use this trick because they used optimization algorithms that were incapable of handling degeneracies. But the iteration which we will propose in the next section does not suffer from this problem.)

THEOREM 3. *There is a 1–1 correspondence between full Steiner topologies on $N \geq 3$ regular sites, and $(N-3)$-vectors $\vec{a}$, whose ith entry a_i is an integer in the range $1 \leq a_i \leq 2i + 1$. As an instant corollary, the number of full Steiner topologies on N regular sites is $1 \cdot 3 \cdot 5 \cdots (2N - 5)$.*

PROOF. We will prove this constructively by induction on N. For $N = 3$, we define the unique full Steiner topology (see Figure 2) to correspond to the null vector! To generate the full Steiner topology corresponding to $(a_1 \cdots a_{N-3})$ from the topology corresponding to $(a_1 \cdots a_{N-4})$, take regular site N and attach it, via a line segment, to the middle of edge number a_{N-3} in the old topology (which has $2N - 5$ edges and $N - 1$ regular sites). The numbering conventions for all the edges and vertices are illustrated in the figure. It is easy to see that each choice of the "sprout" edge leads to a different Steiner topology, and clearly (consider the inverse transformation) every full Steiner topology is generated in this way. □

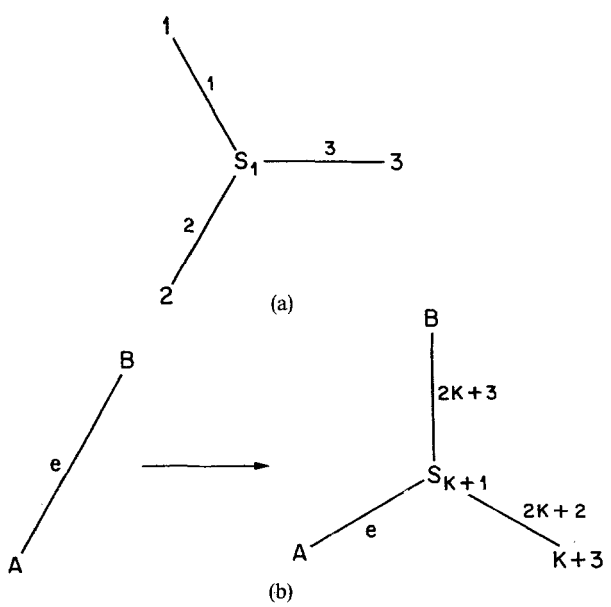


Fig. 2. Edge and vertex numbering conventions for Full Steiner Topologies and corresponding vectors. (a) This tree with three regular sites 1, 2, 3 and one Steiner site S_1 corresponds to the null vector ( ). The edges are also numbered 1, 2, 3. (b) "Sprout" operation showing how to generate the tree represented by a k-vector, from the tree represented by its prefix $(k-1)$-vector. The kth element of this vector is e; edge e of the old tree is modified as shown. Note the introduction of two new vertices, namely, the Steiner point S_{k+1} and regular site $k + 3$, and two new edges, numbered $2k + 2$ and $2k + 3$.

It is now a simple backtracking problem to generate all the topology-describing $(N-3)$ vectors. But there is one additional simple observation which can vastly speed up the generation step.

THEOREM 4. *For any set of N distinct regular sites in any Euclidean space, the length of the shortest tree (interconnecting only the first $N-1$ sites) with the topology (prefix) vector $a_1 \cdots a_{N-4}$ is no greater than the length of the shortest tree with the topology vector $a_1 \cdots a_{N-3}$. (In fact, if the points are in general position, the inequality is strict.)*

PROOF. Consider removing the edge e connecting the last regular site (number N) to the rest of the tree. This shortens the tree and upon optimizing what is left, a further shortening occurs. The only way that equality can hold is if e has length zero—impossible for points in general position. □

This allows us, during the backtracking, to avoid generating all descendants of any bad prefix. During the backtracking, it is worthwhile to sort the vector extensions (in the proof of Theorem 3) into an order such that the shortest trees are considered first. This way, better upper bounds are (usually) found sooner, permitting more pruning using Theorem 4 and thus speeding up the algorithm. The entire algorithm is given in Appendix 1.

We will now discuss a few aspects of the algorithm. First, one must begin the algorithm with some upper bound "STUB" on the length of the Steiner minimal tree. The simplest approach is to use ∞. One could also use the length of the minimum line-segment spanning tree, which may be found in $\mathbf{O}(dN^2)$ time using Prim's algorithm, or indeed the length of any heuristic tree [3], [17], [46], [47], [53]. However, it is my experience that the backtracking algorithm itself soon generates a very good upper bound, so that this idea will not improve the performance substantially.

Also, the algorithm works if the N regular sites are inputted in any order. But perhaps, some orders are better than others. For example, one could presort the points in lexicographic coordinatewise order, use preorder in the Friedman–Finkel–Bentley "optimized k-dimensional tree" [13], use an order generated by depth first search in the minimum spanning tree (or the Euclidean *maximum* spanning tree [39]), or apply a random permutation. All of these ideas sound interesting but I have not done very much experimentation. The only idea which I can definitely recommend is, when using a highly symmetric point set (such as the regular d-octahedron), to select an ordering which will allow the backtracking to be terminated early to take advantage of the symmetry.

One could also incorporate various other more complicated, and powerful, pruning bounds into the backtracking/branch-and-bound framework we have described. One would like to have very powerful bounds, similar to the ones available for the "traveling salesman problem" [21], [22], [23], [35] which are usually accurate to within 1%. Unfortunately, for the d-dimensional Steiner problem, only a few bounds are known; usually they are all quite weak (10%); and they cannot take advantage of a partial specification of the SMT topology

(such as a prefix vector) to improve the bound. For example, a theorem of Graham and Hwang [19] states that in any Euclidean space, the minimum Steiner tree on N regular sites is no shorter than the length of the minimum spanning tree, times $1/\sqrt{3} \simeq 0.577$. (This constant was recently improved by D. Z. Du, to 0.615.) In the plane, Chung and Graham [5] improved this constant to 0.824. A different approach to finding lower bounds is outlined in Problem 2 on p. 330 of this issue.

The only aspect of the algorithm which has not yet been described is how to optimize numerically a prespecified Steiner tree topology, by finding the optimum locations for its Steiner points. This is the subject of the next section.

3. Methods for Optimizing Prespecified Steiner Tree Topologies. This section introduces a simple and generally convergent iteration to find the optimum coordinates of the Steiner points (for any d) in any *prespecified* Steiner topology T (not necessarily "full" or "nondegenerate"). Each application of the iteration takes $\mathbf{O}(N)$ time, the sequence of trees produced have monotonically decreasing lengths, and convergence occurs except from a set of initial trees of measure zero. The speed of convergence is analyzed—it is generally geometric, except in certain degenerate cases (120° angles at regular sites) when the kth iterate has error like $1/k$.

An exact algorithm for optimizing *two-dimensional, full, nondegenerate* Steiner tree topologies was given by Melzak [36] and later improved by Hwang [28] to run in $\mathbf{O}(N)$ time. (We present that algorithm in Section 3.4.) Hwang and Weng [29] have recently devised a more powerful algorithm that works for nonfull or degenerate two-dimensional topologies, but the run time is $\theta(N^2)$ and the method is quite complicated. Unfortunately, no exact algorithm is known in dimensions $d \geq 3$, and indeed until now, no good numerical method was even known.

Gilbert and Pollak [18] had pointed out that optimizing a respecified Steiner topology is a "strictly convex" optimization problem, i.e., the only local optimum is global. Unfortunately, the length function has nonsmooth, complicated behavior at and near the optimum, so that the usual numerical methods are ineffective.

A simple iterative process which never increases the length of a tree is: Pick some Steiner point in the tree and replace it with the optimum Steiner point of its neighbors in the tree. This iteration will always converge to the unique optimum (if all the Steiner points are repeatedly scanned and updated). Unfortunately, the convergence can be very slow, essentially because it can take a long time for "information" to "propagate" along the tree. (The tree rapidly becomes "locally" optimal, but "global" structure is quite irrelevant to it.)

It would be preferable to devise an iteration where, at each step, all of the Steiner points are updated simultaneously and compatibly. (The situation at each iteration is analogous to using a Gauss–Seidel iteration to solve a diagonally dominant, dense, linear system, versus solving the entire system directly in one step.) We will now propose just such an iteration.

Some idea of its superiority over the one-point-at-a-time method may be gained by noting two facts.

1. After just one step of the simultaneous iteration (starting from anywhere), the Steiner points will automatically all lie inside the convex hull of the regular sites (whereas, perhaps, a very large number of one-point steps might be required to assure this, for some unfavorable starting configuration), and will actually form a quite reasonable tree since they will exactly minimize a function that is related to the length.
2. In practice: Typically the simultaneous iteration converged more quickly than several variants of the one-at-a-time method. In all dimensions higher than 2, the time advantage was roughly in the ratio $N/2$, where N was the number of regular point sites, and much greater performance ratios arose when only low accuracy was required. (Typically, convergence to machine accuracy would occur after about 20 simultaneous iterations, although this number was highly variable.) In two dimensions, however, the two methods usually used about the same amount of time! Presumably this was because most optimized two-dimensional trees may be partitioned into small full Steiner trees, so that information need not "travel large distances" to obtain convergence.

3.1. The Iteration, Proof of Convergence, and Implementation. Let the Steiner topology be called T. We will call the set of Steiner points S, and the set of regular sites R. Let the coordinates of the kth site be given by the d-vector $\vec{x}_k$, where $\vec{x}_1 \cdots \vec{x}_N \in R$ and $\vec{x}_{N+1} \cdots \vec{x}_{N-2} \in S$.

THEOREM 5. *As the ith step in an iterative process, $i = 0, 1, \ldots$, solve the system of $N-2$ linear equations*

$$\vec{x}_k^{(i+1)} = \sum_{j \text{ s.t. } jk \in T} \frac{\vec{x}_j^{(i+1)}}{|\vec{x}_j^{(i)} - \vec{x}_k^{(i)}|} \Bigg/ \sum_{jk \in T, j \in S} \frac{1}{|\vec{x}_k^{(i)} - \vec{x}_j^{(i)}|}, \qquad N+1 \le k \le 2N-2.$$

(each equation being d-vector valued) for the unkowns $x_k^{(i+1)}$ where $\vec{x}_k \in S$ represents the coordinates of the kth Steiner point. (The $\vec{x}_k$'s with $1 \le k \le N$ remain fixed.) Then: from all initial choices of Steiner point coordinates $\vec{x}_k^{(0)}$, except for a set of measure zero in $\mathbf{R}^{(N-2)d}$, the iteration converges to the unique optimum Steiner point coordinates that minimize the total tree length L. This convergence happens in such a way that the sequence of tree lengths $L(S^{(i)})$, where

$$L(\vec{x}_{N+1}, \ldots, \vec{x}_{2N-2}) = \sum_{j,k:\, 1 \le j < k \le 2N-2,\, jk \in T} |\vec{x}_j - \vec{x}_k|,$$

is monotonically decreasing.

PROOF. First, it is readily seen that, after the first iteration, all iterates of all Steiner points lie withint the convex hull of the regular sites. The Bolzano–Weierstrass theorem tells us that any infinite sequence of points inside a compact region has some infinite subsequence approaching a limit point. We plan to show that the only Bolzano–Weierstrass subsequence limit point that can exist (except when the initial iterate lies in some set of measure 0), represents the optimum tree.

After each (ith) iteration, $S^{(i+1)}$ actually *exactly* minimizes a certain quadratic form which we will call $Q^{(i)}(S)$:

$$Q^{(i)}(S) = \sum_{jk \in T,\, k \in S,\, j<k} \frac{|\vec{x}_k - \vec{x}_j|^2}{|\vec{x}_k^{(i)} - \vec{x}_j^{(i)}|}$$

$Q(S)$ is related to the length $L(S)$ as follows:

$$L(S^{(i)}) = Q^{(i)}(S^{(i)}), \qquad \nabla L(S^{(i)}) \propto \nabla Q^{(i)}(S^{(i)}).$$

Physically, one may think of Q as the potential energy of a system of ideal springs on the tree edges; the force constant of each spring is proportional to the reciprocal of its (original) length before the start of each iteration. The ith iteration causes all the springs to relax simultaneously to the configuration minimizing the potential energy $Q^{(i)}$.

These springs are ordinary springs obeying Hooke's law, and are not the constant tension "soap films" which are encountered elsewhere in the World of Steiner Trees. The iteration equation of Theorem 5 may be rewritten

$$\sum_{jk \in T} \frac{\vec{x}_k^{(i+1)} - \vec{x}_j^{(i+1)}}{|\vec{x}_k^{(i)} - \vec{x}_j^{(i)}|} = 0, \qquad k = N+1, \ldots, 2N-2,$$

which makes it clear that the forces on the kth Steiner point are in equilibrium. Now write

$$\begin{aligned} L(S^{(i)}) &= Q^{(i)}(S^{(i)}) \geq Q^{(i)}(S^{(i+1)}) \\ &= \sum_{j,k,\, j<k,\, jk \in T,\, k \in S} \frac{(|\vec{x}_k^{(i)} - \vec{x}_j^{(i)}| + |\vec{x}_k^{(i+1)} - \vec{x}_j^{(i+1)}| - |\vec{x}_k^{(i)} - \vec{x}_j^{(i)}|)^2}{|\vec{x}_k^{(i)} - \vec{x}_j^{(i)}|} \\ &= L(S^{(i)}) + 2[L(S^{(i+1)}) - L(S^{(i)})] \\ &\quad + \sum_{j,k,\, j<k,\, jk \in T,\, k \in S} \frac{(|\vec{x}_k^{(i+1)} - \vec{x}_j^{(i+1)}| - |\vec{x}_k^{(i)} - \vec{x}_j^{(i)}|)^2}{|\vec{x}_k^{(i)} - \vec{x}_j^{(i)}|} \end{aligned}$$

In the last expression, the sum is obviously nonnegative, hence

$$L(S^{(i)}) \geq L(S^{(i+1)}).$$

The only way that equality can be achieved is if we are at a fixed point of the iteration. The only fixed points of the iteration are at the optimum (where L is minimized) or at certain places where one or more of the edge lengths in T are zero. (At these places, the iteration as we have written it is not strictly defined since there is a division by zero, but the singularity is removable so we remove it.) However, clearly these nonoptimum fixed points are unstable since an arbitrarily small perturbation of S in any L-decreasing direction will cause the

iteration to resume decreasing L, and by the Gilbert–Pollak L-convexity property mentioned earlier, there is always such a direction—in fact, there is an entire half-space of such directions that go "down." A slightly less obvious fact is that a small step in the *opposite* direction to any Gilbert–Pollak "down" direction also causes the iteration to resume decreasing L. This is because, reasoning physically, each of the "springs" that were of nonzero length are essentially unaffected by a sufficiently small perturbation, while all of the springs of zero length at the fixed point have equal length after *either* a small perturbation, or its negation. So, the relaxed configurations after one iteration are identical—and therefore both decrease the total length L, causing an irreversible departure from the neighborhood of the fixed point in question.

Thus we see that not only are the nonoptimum fixed points unstable, in fact, they are unstable in essentially every direction. (One may show rather easily that the set of stable directions is at least of codimension two, in fact.)

So, there is a 0-measure set of initial iterates which, after a finite number of steps, winds up on a very low-dimensional unstable manifold of nonoptimal fixed points. Disregarding this set, we see that L strictly decreases every iteration. Hence any Bolzano–Weierstrass subsequence limit tree must be a fixed point of the iteration (by continuity of L and the iteration function, and since the only points where the iteration does not decrease L are fixed points). But we have ruled out all fixed points except for the optimum. □

REMARK. Of course, since computers unfortunately must do finite precision arithmetic, on a real-life computer there is indeed a nonzero probability of convergence to a nonoptimum fixed point. However, because of the extreme instability properties of such points, this possibility may be defeated by applying a small random perturbation (or one could use the steepest descent direction $-\vec{\nabla}L$) if necessary. (Also see the discussion of convergence detection in Section 3.3.)

REMARK. The iteration is invariant under scaling, rotation, and reflections of space.

THEOREM 6. *There is an* $\mathbf{O}(Nd)$*-flop*[2] *algorithm which solves the system of equations occurring in (i.e., performs) one iteration of Theorem 5.*

PROOF. Solve the linear system by "Gaussian elimination on leaves of a tree." To be more specific, a graph structure may be associated with any linear system $A\vec{x} = \vec{b}$, namely, there is a graph edge ij iff $A_{ij} \neq 0$. One may "eliminate" a particular variable x_i to obtain a smaller linear system for the other x's, plus an extra equation expressing x_i in terms of the other x's. The graph for the new

[2] A "flop" is a FLoating point OPeration, consisting of an array access, an addition, and a multiplication or division of real numbers.

linear system is the same as the old graph, except: vertex i and its incident edges are gone, and extra edges are added, if necessary, so that i's old neighbors are interconnected by a clique. Thus eliminating a *leaf* of a tree results in another tree with one less vertex. By keeping track of all valences in the tree and maintaining the leafs in a queue, we may perform all eliminations in $\mathbf{O}(N)$ total time, obtaining a tree with only one vertex at the end. Each time a leaf is eliminated, it (and its backsubstitution equation) is stored on a stack. We may then perform all backsubstitutions by popping the stack.

Actually, in our case there are d linear systems, each of size $(N-2) \times (N-2)$, and each having the same graph structure. We solve them all in $\mathbf{O}(Nd)$ time. Detailed code is in the routine "optimize" in Appendix 1. □

This algorithm generalizes the more widely known result that tridiagonal systems may be solved in $\mathbf{O}(N)$ time.

3.2. Analysis of Speed of Convergence. We have now proved convergence to optimality—the remaining question that confronts us is: How quickly does convergence occur?

To answer this question, let us first analyze the special case of the full Steiner topology on three regular sites that form an isoceles triangle of apex angle θ. This case is sufficiently simple that it may be completely analyzed, yet is sufficiently interesting that it apparently exemplifies all the kinds of convergence behavior that arise in the general problem.

We will assume that the initial guess for the Steiner point location $\vec{x}^{(0)}$ is chosen at infinity.

THEOREM 7. *In the situation described above, convergence to the Steiner point will always occur. If the ith iterate approximation to the Steiner point $\vec{x}^*$ is $\vec{x}^{(i)}$, $i \geq 1$, and the length of the Steiner minimum tree is L, then the "scaled error" $s^{(i)}$ at the ith iteration, defined by $s^{(i)} \equiv (\vec{x}^{(i)} - \vec{x}^*)/L$, is bounded by:*

$$(1) \quad s^{(i)} < \left(\frac{1+\sqrt{3}\tan(\theta/2)}{4}\right)^i \qquad \text{if} \quad \theta \leq 120°.$$

$$(2) \quad s^{(i)} < \left(2\cos\left(\frac{\theta}{2}\right)\right)^i \qquad \text{if} \quad 120° < \theta < 180°.$$

$$(3) \quad s^{(i)} < \frac{1}{3i} \qquad \text{if} \quad \theta = 120°.$$

PROOF. *Case* 1 (*and* 3). Without loss of generality, let the triangle's vertices (the regular sites) have XY-coordinates $(-1, 0)$, $\frac{1}{2}(R, \pm\sqrt{3}R)$, so that the Steiner point (which the iteration will converge to) is at $(0, 0)$, and the Steiner minimum tree has edge lengths 1, R, and R, and total length $L \equiv 2R + 1$. The first iteration will set $\vec{x}^{(1)}$ to the triangle's centroid, namely $((R-1)/3, 0)$, and all later iterates $\vec{x}^{(i)}$ will lie on the axis of symmetry $Y = 0$. Hence, we only need to consider the successive X-coordinates $x^{(i)}$ and how quickly they converge to 0. In our special

case, the iteration may be written as

$$x^{(i+1)} = \frac{(x^{(i)}+1)R - K}{K + 2(x^{(i)}+1)}, \qquad K \equiv \sqrt{\left(\frac{R}{2} - x^{(i)}\right)^2 + \tfrac{3}{4}R^2},$$

which has Taylor expansion

$$x^{(i+1)} = \frac{2R+1}{2R+4}\, x^{(i)} - \frac{12R^2 + 9R + 6}{8R^3 + 32R^2 + 32R}\,(x^{(i)})^2 + \cdots.$$

Therefore, there is "linear" convergence, with convergence ratio

$$C_\theta = \frac{2R+1}{2R+4} = \frac{1 + \sqrt{3}\tan(\theta/2)}{4}.$$

Indeed, upon defining the "dimensionless" version $s^{(i)}$ of $x^{(i)}$ by $s^{(i)} \equiv x^{(i)}/L$, one may show

$$s^{(i+1)} < (C_\theta)^i.$$

for all $0 < R < \infty$, i.e., $0 \le \theta < 120°$, $i \ge 1$.

If $\theta \to 120°$, we have $R \to \infty$, and then

$$s^{(i+1)} = s^{(i)} - 3(s^{(i)})^2 + \mathbf{O}((s^{(i)})^3)$$

so that $s^{(i+1)} < 1/3i$.

Case 2. Let the three vertices have coordinates $(0, 0)$, $(1, \pm R)$, where $R > \sqrt{3}$. The "Steiner point" is then the triangle apex $(0, 0)$, the length of the Steiner minimal tree is $L \equiv 2\sqrt{R^2+1}$, and the centroid is $\vec{x}^{(1)} = (\frac{2}{3}, 0)$. Since all later iterates lie on the line $Y = 0$, we restrict our interest to the X-coordinate of the ith iterate, which we will call $x^{(i)}$. We defined $x^{(i)}$ so that it converges to 0. The iteration may be written

$$x^{(i+1)} = \frac{x^{(i)}}{K + x^{(i)}}, \qquad K \equiv \tfrac{1}{2}\sqrt{R^2 + (1 - x^{(i)})^2}$$

with Taylor expansion

$$x^{(i+1)} = \frac{4}{L}\, x^{(i)} - \frac{16(L-1)}{L^3}\,(x^{(i)})^2 + \mathbf{O}\left(\frac{(x^{(i)})^3}{L^3}\right).$$

The convergence is once again linear, with the "scaled error" $s^{(i)} \equiv z^{(i)}/L$ obeying

$$s^{(i+1)} < (C_\theta)^i, \qquad C_\theta = \frac{4}{L} = 2\cos\frac{\theta}{2}. \qquad \square$$

REMARK. Roughly speaking, if the triangle's shape is "bounded away" from a 120° triangle, then a constant number of iterations will suffice to halve the error. For any triangle, $1/\varepsilon + 1$ iterations will yield relative accuracy $< \varepsilon$, but generally the performance is much better.

Let us now discuss the asymptotic rate of convergence of the iteration in the general case, when we have a general full Steiner topology (possibly with degenerate optimum) on N regular sites in d-space. Let the optimum locations of the Steiner points be called S^*. Let the "error" at the ith iteration be the L_∞ norm of $S^* - S^{(i)}$, regarded as a $d(N-2)$-vector.

If the sites are in general position, then geometric convergence will occur, since the Taylor expansion of the iteration about the fixed point will generally have a linear term that is a nonzero matrix with all eigenvalues of magnitude strictly less than 1. Certain degenerate configurations of regular sites (such as a triangle with a 120° angle, in the example above) do exist, though, that cause sublinear convergence.

I believe that the rate of convergence is always such that at most $\mathbf{O}(N/\varepsilon)$ iterations are required to get the length within ε of optimal, and usually the performance is much better. Arguments concerning the condition number of the quadratic forms $Q^{(i)}$ may be made, which support this belief.

An additional remark is that, since the rate of convergence is generally geometric, standard techniques [16] for accelerating this convergence could be applied. I have not experimented with this idea.

3.3. Detection of Convergence. We have now proved that convergence of the iteration of Theorem 5 always occurs, and an analysis of the speed of convergence seems to indicate satisfactory performance will be obtained. However, there is still the question of detecting when convergence has occurred, so that we may decide to stop.

It would be wrong to take the obvious, but dangerous, course of testing whether one iterate differs substantially from the next—and if it does not, to decide that "convergence" has occurred. Some experiments convinced me that a better idea was to use the fact that a necessary and sufficient condition for optimality is that all angles in the tree are $\geq 120°$. This suggests defining the "error figure" E of a tree T by

$$E^2 = \sum_{\substack{i \in S \\ ij \in T \\ ik \in T \\ j \neq k}} \operatorname{pos}(2(\vec{x}_j - \vec{x}_i)\cdot(\vec{x}_k - \vec{x}_i) + |\vec{x}_j - \vec{x}_i|\cdot|\vec{x}_k - \vec{x}_i|),$$

where $\operatorname{pos}(x) \equiv \max(x, 0)$, and terminating the iteration when $E \ll L$.

3.4. The Hwang–Melzak Algorithm. The following algorithm, given $N \geq 5$ "regular" point sites in the plane and a "full Steiner topology" T interconnecting them, will find the optimum coordinates for the Steiner points in $\mathbf{O}(N)$ time, assuming that the topology is "nondegenerate," i.e., no edge length in the optimum configuration is zero.

HM1. Find the valence of every vertex, find a leaf F, call F the tree "root," and set up the tree edges as parent $\rightarrow$ child and child $\rightarrow$ parent pointer adjacency lists.

HM2. Find the generation-number of every vertex in the tree, by breadth-first search [49]. (We now have two N-element arrays allowing one to read off the valence and generation number of vertex i in $\mathbf{O}(1)$ time.) Create a sorted doubly linked list L of the leaves of T, sorted by generation number. (This sorted list may be generated for "free" as a byproduct of the breadth-first search.)

HM3. Find a leaf A in the youngest generation by looking at the last element of L. $S_1 = father(A)$. $S_2 = father(S_1)$. Sons of S_1 are $\{A, B\}$. Sons of S_2 are $\{S_1, V\}$.

HM4. If V is a leaf, then: delete $\{A, B\}$ and their incident edges from T and L, create a new regular site K at the foreign (i.e., not A and not B) vertex of the equilateral triangle with side AB (lying on the opposite side of line AB from V), attach K as a new child of S_1 in T, and insert it at the last position in L.

HM5. Otherwise [V not a leaf]: Let the two children of V be C and D. Since V belonged to the next-to-youngest generation, they are leaves. If A and B lie on the same side of line CD: delete $\{C, D\}$ and their incident edges from T and L, create a new regular site K at the foreign vertex of the equilateral triangle with side CD (lying on the opposite side of line CD from A), attach K as a new child of V in T, and insert it at the last position in L. Otherwise: delete $\{A, B\}$ and their incident edges from T and L, create a new regular site K at the far corner of the equilateral triangle with side AB (lying on the opposite side of line AB from C), attach K as a new child of S_1 in T, and insert it at the last position in L.

Every time a pair of adjacent leaves $\{A, B\}$ is "deleted" from T, it is pushed onto a stack along with the name of their common parent P.

HM6. If T has ≥ 5 leaves (i.e., if $|L| \geq 5$), go back to Step 3.

HM7. Optimize T (which has ≤ 4 leaves) by brute force.

HM8. Backpedal through the reduction history, by: successively popping triples A, B, P off the stack, and replacing the line segment $P - son(P)$ by the three line segments $\overline{PS}$, $\overline{SA}$, $\overline{SB}$, where S is the Steiner point of ΔABP.

It should now be clear how to implement the algorithm to run in $\mathbf{O}(N)$ time. For a proof of correctness, see Hwang [25] and Melzak [36]. Melzak's original algorithm ran in exponential time; Hwang improved this to $\mathbf{O}(N)$ time but omitted algorithm details.

4. Some Steiner Trees Found with the Backtrack Algorithm. The algorithm was implemented in C [31] on a VAX and an IRIS workstation. The code is given in full in Appendix 1.

The Steiner minimal trees for the vertices of most of the regular and Archimedean d-polytopes with ≤ 16 vertices were found and are given in the tables. Here

Table 1. SMT for vertices of icosahedron (actual computer output).

Input	Output
1 2 3	new record length 18.5529
1 0 1.6180339887	topology-describing vector
0 1.6180339887 1	2 3 7 6 7 6 14 12 18
1.6180339887 1 0	Steiner point coords
−1 0 1.6180339887	0.550563 −0.0318824 1.2052
0 1.6180339887 −1	−0.0714358 0.418004 1.27697
1.6180339887 −1 0	1.12641 0.325618 −0.302838
1 0 −1.6180339887	1.08946 −0.423524 0.144941
0 −1.6180339887 1	0.747616 0.398184 −0.935115
−1.6180339887 1 0	0.604706 −0.514643 0.621463
−1 0 −1.6180339887	−0.0607402 0.970873 −0.965593
0 −1.6180339887 −1	−0.750714 0.654759 −0.948811
−1.6180339887 −1 0	0.0320559 −1.20717 0.554933
	−0.419159 −1.27778 −0.0704503
	edges

1 13; 2 14; 3 15; 4 14; 13 14; 5 19; 13 18; 6 16; 15 16; 7 17; 15 17;
8 21; 16 18; 9 20; 17 19; 10 20; 19 20; 11 22; 18 21; 12 22; 21 22;
done

is the actual input file (suitable for use with the program given in Appendix 1) and (a slightly edited subset of) the actual computer output, for finding the SMT for the twelve vertices of a regular icosahedron in $\mathbf{R}^3$.

The final SMT length was 18.5529, while the MST length is 22, for a ratio of 0.843314. It was conjectured by Gilbert and Pollak [18], and later in [4], that

$$\rho_d = \inf_P \frac{\text{length}(\text{SMT}(P))}{\text{length}(\text{MST}(P))}$$

(where the infimum is taken over all point sets P in $\mathbf{R}^d$) was exactly the value attained by using $P =$ the vertices of a regular d-simplex.

THEOREM 8. *The Gilbert–Pollak conjecture is false if* $3 \leq d \leq 9$.

Table 2. SMT/MST ratios for the vertex sets of various d-polytopes.

d	Reg. simplex	Reg. octahedron	Other
2	0.866025	0.910683	reg. pentagon 0.972789; all other regular polygons 1 [11]
3	0.813052	0.811197	reg. icosahedron 0.843314; cube 0.885165
4	0.783748	0.768719	10-vertex Archim. Polytope 0.785599; 4-cube 0.874957
5	0.764564	0.745742	16-vertex "Half cube" polytope ≤0.752291
6	0.751427	0.731990	
7	0.741264	≤0.722478	
8	0.733767	≤0.715502	
9	0.727434	≤0.711125	
10	0.722504		(Note: In cases where the search space was too large for
11	0.718118		an exhaustive search, only an upper bound is given.)

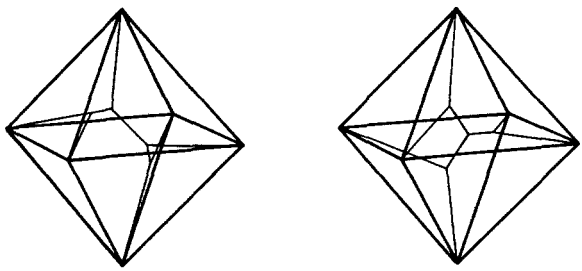


Fig. 3. (left) Correct and (right) incorrect SMTs for the regular 3-octahedron.

PROOF. It is apparent from Table 2 above that for $3 \leq d \leq 9$ (and presumably in all higher dimensions as well) that the conjecture is false, since the regular octahedron has a smaller ratio than the regular simplex. Also, observe that in five dimensions the half-cube is a counterexample too. □

Gilbert and Pollak [18] had not noticed this for $d = 3$ because they had computed an incorrect SMT for the octahedron! Pictures of the correct and incorrect trees are given in Figure 3. The correct tree, with length $\simeq 5.73603$, is about 1% shorter than Gilbert and Pollak's tree, which has length $3(1 + \sqrt{3})/\sqrt{2} \simeq 5.79555$. It is interesting that the optimum tree has essentially no symmetry, while the two "obvious" attempts with considerable symmetry are not optimal. This (and some other examples!) shows that finding SMTs is a tricky business with many pitfalls. When working without an exhaustive computer search, one is dangerously likely to be led astray by one's geometric intuition. It still seems likely that the Gilbert–Pollak conjecture in the plane, namely that $\rho_2 = \sqrt{3}/2$, is true.[3] It is known that any counterexample must have ≥ 5 Steiner points; and it has also been shown [5] that $0.824168 \leq \rho_2 \leq \sqrt{3}/2 \simeq 0.866025$.

To find the SMT for the regular d-simplices S_d, I embedded them in a $(d + 1)$-dimensional space by using the ith row of the $(d + 1) \times (d + 1)$ identity matrix as the coordinate of the ith vertex. (The MST of S_d has length $d\sqrt{2}$.) This embedding has the advantage that the coordinates of the Steiner points in $\mathbf{R}^{d+1}$ are also their barycentric coordinates. The Steiner minimal trees for $S_2 \cdots S_8$ are summarized in Table 3 (in Appendix 2). These Steiner trees were also studied by Chung and Gilbert [4] without a computer. Their trees were optimal for $d = 2 \cdots 11$, *except* that our result for S_8 improves theirs.

It is easy to see that SMT(S_d) always has a full topology. When specifying the topologies of the SMTs for S_d, it is clearly only necessary to give the "skeleton" of the trees (i.e., the part defined by the Steiner points alone after removing the leaves), since all the simplex vertices are equivalent. Upon drawing the skeletons of all the topologies of the optimum SMTs for S_d, $d = 1 \cdots 11$, I found an interesting fact.

[3] Added in proof, D. Z. Du and F. K. Hwang have recently proven the Gilbert–Pollak conjecture in the plane, according to *FOCS*, **31** (1990), 76–85.

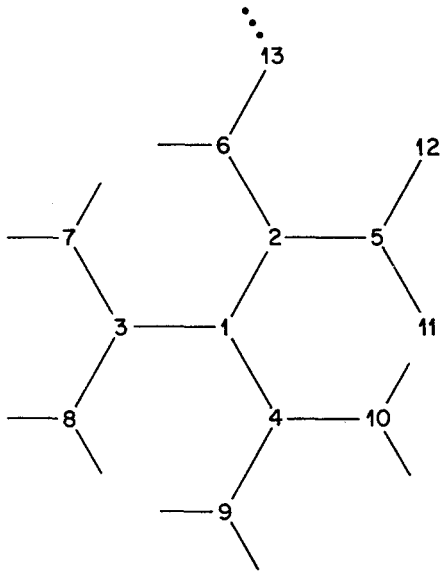


Fig. 4. Topologies for skeletons of SMTs of regular d-simplices, $d = 1 \cdots 11$; delete vertices with numbers $\geq d$ (and their adjacent edges) from the figure. (The "skeleton" of a tree topology is that tree-topology with all leaves removed.)

FACT. *The skeleton of the SMT for the vertices of the regular d-simplex,* $d = 2 \cdots 11$, *is given by removing all the vertices with numbers* $\geq d$ *(and their adjacent edges) from the infinite spirally labeled Bethe-tree in Figure* 4.

CONJECTURE 2. *This fact is also true for all dimensions* $d \geq 11$. *In particular, for all* $d \geq 2$ *the topology for* $SMT(S_d)$ *is a subtree of the topology for* $SMT(S_{d+1})$. *Another consequence of this conjecture is that the trees of Chung and Gilbert are optimal if d is of the form* $d = 3 \cdot 2^P$, *and that the limiting SMT/MST ratio is* $\sqrt{3}/(4 - \sqrt{2}) \simeq 0.66984$ *as* $d \to \infty$.

The vertices of the canonical regular d-octahedron G_d are the $2d$ vectors with one coordinate ± 1 and the others zero. (The length of the MST is $(2d - 1)\sqrt{2}$.) The Steiner minimal trees for $G_3 \cdots G_6$ are summarized in Table 4 (Appendix 2).

The topologies of these SMTs are shown in Figure 5. (Notation: "n" represents the vertex with coordinate $x_n = 1$; "$\bar{n}$" represents the vertex with coordinate $x_n = -1$.) It is too early to speculate about what the pattern is here, but whatever it is, it is plainly more interesting and mysterious than the one for the regular d-simplices. The regular unit d-cube has 2^d vertices. The coordinates of the ith vertex are the binary expansion of $i - 1$. The SMTs for $C_1 \cdots C_4$, found by computer, obey the following formula:

CONJECTURE 3. *The Steiner minimum tree for the vertices of the unit d-cube is of length* $(2^{d-1} - 1)\sqrt{3} + 1$ *for each* $d \geq 1$.

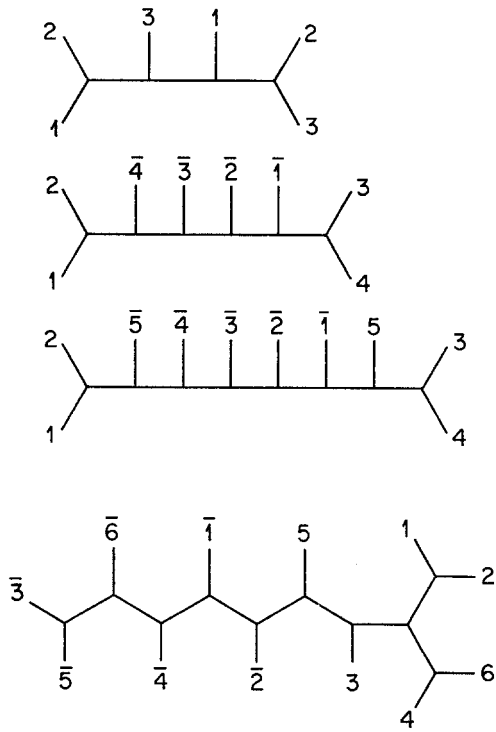


Fig. 5. Topologies for SMTs of regular d-octahedra, $d = 3 \cdots 6$ (at top, $d = 3$, at bottom $d = 6$).

This is rather easily shown (by induction on d) to be the length of the optimized FST whose topology is the complete binary tree with 2^d leaves, and one root. (This "root" will actually lie at the midpoint of an SMT *edge* and will be located at the center of the d-cube.) Explicit coordinates for C_2 and C_3 are tabulated in Table 5 (Appendix 2). I also computed some SMTs and putative SMTs for the vertices of various other Archimedean d-polytopes.

The four-dimensional polytope with 10 vertices whose 5-coordinates are the permutations of $(2, 2, 2, -3, -3)$ has MST length $45\sqrt{2} \simeq 63.6396$. The SMT has length 49.9952 and the Steiner point coordinates are given in Table 6 (Appendix 2).

The "half-cube" in five dimensions, whose 16 vertices have coordinates that are all 5-bit binary words containing an even number of 1's, has MST length $15\sqrt{2} \simeq 21.2132$ and SMT length ≤ 15.9585. Steiner point coordinates for a tree achieving this bound are in Table 7 (Appendix 2).

Part II: Algorithms for Steiner Minimal Trees in the Plane

We will now shift our attention from the realm of the practical and the simple to more esoteric and theoretical results. The fundamental problem is: What is the asymptotic computational complexity of finding trees for N sites in d-space, with length either exactly optimal or $\leq (1 + \varepsilon) \times$ optimal?

We stick to the "real RAM" model of computation, which allows infinite-precision operations with real numbers; we assume that solving any algebraic equation of bounded degree may be accomplished in $\mathbf{O}(1)$ time. This is for simplicity—and also in order to dodge difficult problems involving comparing two algebraic numbers (in particular, sums of square roots). The method given in Part I runs in $\mathbf{O}(2^N N!)$ steps. Here each "step" is a topology-optimization, taking approximately $\mathbf{O}(N/\varepsilon)$ time, and usually more like $\mathbf{O}(N \log(1/\varepsilon))$, to converge to within ε.

There is a simple alternative method, which could run faster if N and ε are large, and d is small. It is based on noticing that any Steiner tree topology contains a "center," a vertex whose largest subtree (removing the center vertex splits the tree topology into ≤ 3 subtrees) contains $\leq 2N/3$ of the regular sites [30]. The leads to a "divide and conquer" algorithm.

DC1. Consider all 2^N ways to divide the N regular sites into two subsets. For each subset, *recursively* find the shortest tree interconnecting: the sites in a subset, and an additional "glue" point Q. (Which is just some point of space—not necessarily a site.) We try $(N\sqrt{d}/\varepsilon)^d$ positions for Q (say, at the points of an appropriately scaled cubic lattice which lie inside the smallest axis-parallel d-cube that encloses all the regular sites).

DC2. The shortest union of the trees of a subset and its complement (using the same glue point for each), found in this way, has length $\leq (1 + \varepsilon) \times$ optimal.

The total run-time $T(N)$ obeys the recurrence

$$T(N) = 2^N \cdot 3 \cdot (N\sqrt{d}/\varepsilon)^d \cdot T(1 + 2N/3)$$

with solution

$$T(N) = 8^N \mathbf{O}(N\sqrt{d}/\varepsilon)^{d \log_{1.5} N}.$$

Note that if d is fixed and $1/\varepsilon$ grows only polynomially with N, then this algorithm is exponential, rather than superexponential, as are all previous algorithms for SMTs. We will now restrict our attention to the Euclidean plane ($d = 2$), where further improvements are possible.

5. Algorithms for Steiner Minimal Trees in the Euclidean Plane. We present for the first time an algorithm provably running in $\mathbf{O}(C^N)$ time to find the SMT for N point sites in the plane.

We rely on techniques in [44] combined with the following:

LEMMA 1. *For any set of N point sites in the plane, there is a crossing-free line segment directed Euler cycle, consisting of N directed line segment edges (with no two edges in the cycle crossing, but possibly an edge is used twice = once in each*

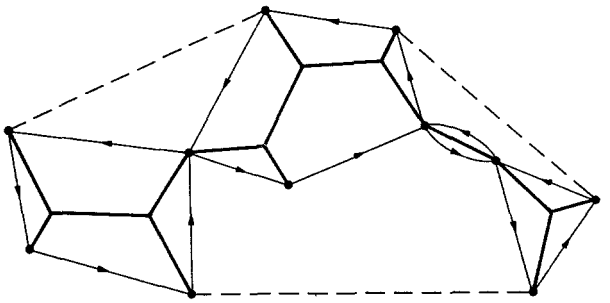


Fig. 6. Directed crossing-free line segment Euler tour surrounding a Steiner minimum tree of 11 sites in the plane, constructed as in the proof of Lemma 1. (Convex hull is also shown with dashed lines.)

direction) on the set such that the Steiner minimal tree of the set lies totally on or inside the cycle.[4]

PROOF (see Figure 6). The Steiner minimal tree divides the convex hull CH into some "polygons." If we walk counterclockwise around the perimeter of any of these polygons, all the Steiner points are left turns (convex vertices) but some of the regular sites may be right turns (reflex vertices). Now inside any simple polygon, there is a crossing-free Hamilton line segment tour that visits all of its reflex vertices and any desired subset (and only this subset) of the convex vertices, as is easy to see. (For example, by drawing disjoint open line segments between the reflex vertices, any simple polygon may be partitioned into convex subpolygons. A suitable Hamilton tour may now be constructed by gluing together convex polygonal chains inside each subpolygon.) The subset we select is the sites, so that no Steiner points are included in the Hamilton polygon. The union of all the Hamilton polygons is obviously a crossing-free line segment directed Euler tour. □

The number of *ordered* full Steiner tree topologies on N regular point sites (with circular ordering prespecified according to some fixed Euler tour) is the same as the number of edge rooted planar triangulations of a convex N-gon, by planar duality. These are the Catalan numbers

$$\frac{1}{N-1}\binom{2N-4}{N-2} \sim \frac{4^N}{16N\sqrt{\pi N}}.$$

[4] In this paper, an "Euler" tour of N sites in the plane is a closed path that visits very site at least once. A "Hamilton" tour is a path that visits every site exactly once. We are *only* allowing polygonal tours composed solely of intersite line segments. A "directed line segment Euler tour" is an Euler tour made of inter-site directed line segments, with no directed line segment ever being reused. "Crossing free" means the interiors of the line segments defining the tour are disjoint. Example: any Euler tour is a union of Hamilton tours on site-subsets.

Gilbert and Pollak had to try all possible leaf orderings and hence took super-exponential time. But we now see that we need only try the node orderings corresponding to the clockwise order around all possible CRLS (i.e., "crossing-free line segment") Euler directed cycles. There are at most C^N such CRLS cycles for some constant $C < 10^{13}$, as was proved by Ajtai *et al.* [2]. This upper bound was considerably strengthened by this author [44].[5] Further, we may generate them all in exponential time, as was also proved in my Ph.D. thesis. (For example, the straightforward backtracking algorithm works.) Also, standard algorithms may be used to generate all ways of triangulating a convex polygon, in a time depending linearly on the size of the output. Therefore:

THEOREM 9. *We may generate all possible topologically valid planar-ordered Steiner tree topologies on a N point planar set in exponential time C^N for some constant C, and using linear workspace. Upon optimizing each one using a polynomial-time exact algorithm* [29], *we may also find the Steiner minimal tree in this time and space bound.* □

If the point set is convex, we may use the convex hull itself as the only candidate (Hamilton) cycle. This leads to a particularly simple exponential ($\simeq 2^N$) time algorithm to find the SMT of a convex planar point set.

Subexponential Algorithm for the SMT of N Cities in the Plane? In the author's Ph.D. thesis [44] was given the first subexponential ($N^{O(\sqrt{N})}$-time, $\mathbf{O}(N^2)$-space) algorithm for a traveling salesman tour in the plane. The obvious question is: Can we extend the techniques which produced the subexponential traveling salesman algorithm to work for the SMT problem? The answer so far is "partially." Before presenting these results, we need to survey briefly "graph separator theorems" and k-set theory.

5.1. Separator Theorems

THEOREM 10 (Miller [37]). *Any maximal planar graph on N vertices, $N \geq 16$, with nonnegative real vertex "weights" summing to W, has a "simple cycle separator" (i.e., a simple cycle subgraph) C such that the weight of the vertices on either side of the separator is $\leq 2W/3$, and the number of vertices on C is $\leq \sqrt{8N}$.* □

Suppose that we triangulate (by line segments with endpoints that are SMT vertices) the vertices of the Steiner minimal tree on N regular sites in the plane in any manner that includes the line segments of the SMT and the convex hull as edges. This creates a planar graph with $\leq 2N - 2$ vertices. In fact, if we add a "point at infinity" and draw rays emanating from the vertices of the convex hull of the sites out to infinity, then we obtain a *maximal* planar graph G, having $\leq 2N - 1$ vertices. Hence by Theorem 10:

[5] This problem was recently also studied by Ryan Hayward, in an article to appear in *Discrete and Computational Geometry*.

THEOREM 11. *Given any N point sites in the plane there is a simple L-gon Q, with $L \leq 4\sqrt{N}$ (possibly one of its vertices is at infinity—in which case the polygon is really a polygonal path), such that $\leq 2N/3$ of the regular sites lie on each side of Q, and such that the Steiner minimal tree crosses (or even intersects without crossing) Q at $\leq 4\sqrt{N}$ places.* □

REMARK. Points closely spaced along a spiral curve form a nice counterexample to some putative strengthenings of this theorem.

We will refer to points where the SMT crosses the separator polygon Q as "glue points," since they glue the two halves of the SMT together.

5.2. k_L Sets. There are 2^N subsets of N objects. However, if there are N distinct point sites in the plane, the number of "k-sets," i.e., site-subsets which may be separated from their complements by a line, is $\leq \binom{N}{2}$. (See [12].) Now define a "k_L-set" to be a subset of N distinct point sites in the plane, which may be separated from the rest by a simple L-gon (possibly with one vertex at infinity). In particular, "k_1-set" and "k-set" mean the same thing, while if $L = N$, one may generate all 2^N subsets as k_L-sets.[6]

THEOREM 12. *The number of k_L-sets is $\mathbf{O}(N^{2L})$ uniformly (i.e., the implied constant does not depend on L, nor on the location of the points), and indeed a linear space algorithm exists that will find them all in $\mathbf{O}(N^{2L})$ time.*

PROOF. All k_L sets may be generated by only using line segments defined by the complete graph K_N on the N sites, plus a site at infinity, and slight perturbations. (There are only 2^L possible perturbations and $\leq \binom{N}{2}^L$ possible choices of L of the edges of K_N.) □

5.3. Algorithms and Time/Space Analysis.

THEOREM 13. *An algorithm exists which, given N point sites in the plane, finds a tree that interconnects them, which is at most $1 + \varepsilon$ times longer than the Steiner minimal tree. The running time of the algorithm is $(N/\varepsilon)^{\mathbf{O}(\sqrt{N})}$. The space usage of the algorithm is $\mathbf{O}(N)$.*

PROOF. Before reading this proof, one should understand algorithm DC given just before the start of Section 5. A similar idea is used here, but it is complicated by the additional need to use k_L-sets and Miller's separator theorem. (Perhaps the most detailed description of the idea is in a forthcoming paper of the author's on

[6] By slightly perturbing the traveling salesman tour so that the desired subset lies outside it.

a subexponential algorithm for the planar traveling salesman problem [45]. The same idea also yields a subexponential algorithm for optimum triangulation in the plane [44, Section IV.A.4.2].)

First, without loss of generality, assume the smallest square which encloses all the sites is the unit square. Then the SMT has length $\mathbf{O}(\sqrt{N})$ and $\mathbf{\Omega}(1)$ so that we need only find a tree with length $\mathbf{O}(\varepsilon)$ + optimal. We will actually solve a somewhat more general problem: Finding Steiner trees with "wormholes." Namely: Given a set of point sites in the plane, and also a set of pairs of these sites which are regarded as preconnected (via "wormholes in space") find the (approximately) shortest network interconnecting all the sites, where we are allowed to use the "preconnections."

The algorithm will depend on Theorems 11 and 12. We find all possible $k_{(4\sqrt{N})}$-sets of regular sites using Theorem 12. At least one of them must be part of the SMT in the sense of Theorem 11. (Note: Theorem 11 still holds for SMTs with "wormholes" because the Steiner tree becomes a forest if [invisible] preconnections are used.) In Theorem 11, there are also $\leq 4\sqrt{N}$ places where the SMT crosses the separator polyline. For our purposes we may assume that each of these places is a point on the lattice $(\varepsilon/N)\mathbf{Z}^2$ which lies inside the unit square. There are thus $(N/\varepsilon)^{\mathbf{O}(\sqrt{N})}$ sets of points we must try (for each guessed subset of the sites) to be sure of finding a correct set of crossing points and also at the same time, a correct SMT node-split. Also, even if we know the node-split and the glue points, we still do not know the ordering of the glue points along the separator path/cycle, so we try all $\mathbf{O}(\sqrt{N})!$ possible orderings.

So, by exhaustive guesswork, we have managed to find some subset of the sites, of cardinality $\leq 2N/3$, which forms a part of the SMT that is separated from the part of the SMT containing the complement site-subset, by some unknown separator polyline, containing $\mathbf{O}(\sqrt{N})$ known glue points in a known order where the SMT crosses the separator. The part of the SMT on the other side of the separator polyline is unknown, but its topological structure (i.e., which of the glue sites it interconnects, and how) may be guessed by trying all forests with $\mathbf{O}(\sqrt{N})$ labeled nodes—namely $N^{\mathbf{O}(\sqrt{N})}$ guesses. We convert this topology-guess into a set of wormholes and solve the problems recursively on each site subset. For any subset of size ≤ 300, however, we solve the recursive problem by brute force consideration of all possible topologies.

The total time usage obeys the recurrence

$$T(N) = \left(\frac{N}{\varepsilon}\right)^{\mathbf{O}(\sqrt{N})} \cdot T(2N/3 + \mathbf{O}(\sqrt{N})), \qquad N > 300,$$

with solution $T(N) = (N/\varepsilon)^{\mathbf{O}(\sqrt{N})}$.

Keeping track of the wormholes and checking that every recursive problem, forest, etc. is actually planar (if it isn't, we throw it out and terminate the recursion) are polynomial-time bookkeeping tasks which may be accomplished in linear space [24]. □

THEOREM 14. *An algorithm exists which, given N point sites in the plane, finds their rectilinear Steiner minimal tree (RSMT) (i.e., the shortest electrical interconnect with distances measured in the L_1 norm). The running time of the algorithm is $N^{\mathbf{O}(\sqrt{N})}$. The space usage of the algorithm is $\mathbf{O}(N)$.*

PROOF. The proof is essentially the same as the one above, but by the use of Hanan's theorem [20], we avoid any reference to ε. Hanan's theorem states that the RSMT is contained within the network of horizontal and vertical lines drawn through all the sites. Therefore, we may assume that each SMT-separator crossing point ("glue point") lies at one of the N^2 vertices of this grid—instead of at one of N^2/ε^2 lattice points, and the result will be the actual exact RSMT, not an approximation. (The graph obtained by triangulating the line segments of the RSMT and the convex hull, and an extra site at infinity, is still planar and triangulated, hence Miller's theorem (10) is still applicable.) □

THEOREM 15 (Provan [42]). *An algorithm exists which, given N point sites in the plane which lie on their own convex hull (indeed, even on their own "Steiner hull" [43]), finds a tree that interconnects them, which is at most $1 + \varepsilon$ times longer than the SMT. The running time of the algorithm is $\mathbf{O}(N^6/\varepsilon^4)$.*

PROOF. Because we will later use a modification of Provan's method, we will present an abbreviated version of his proof here. (The essential idea behind this proof dates to [1].) We will not prove the result for Steiner hull (which notion would take too much space to define and use here), but only for the statement with convex hull.

Call the set of N input sites K. Without loss of generality, assume the smallest square containing all the sites is the unit square. First, the SMT is an *ordered* tree with leaf ordering given by the order of the sites around their convex hull CH. We will regard the N sites as being given in this circular order without loss of generality, and we will take indices modulo N in what follows. We will use intervals $[a, b)$, $a, b \in \mathbf{Z}_N$ to refer to the set of sites with indices lying in that interval. Define L to be the set of points lying on the lattice $\varepsilon/N \cdot \mathbf{Z}^2$ and interior to CH, and also including K. Define $\mathrm{SMT}_L(P)$ to be the minimum Steiner tree of point set P, subject to the constraint that all the Steiner points are in the set L.

Now, for each (circular) interval $[a, b)$, and each point $\vec{v} \in L$, define $C(\vec{v}, [a, b))$ to be the length of $\mathrm{SMT}_L([a, b) \cup \vec{v})$. Thus the cost of the SMT of the entire N-point set K is $\leq (1 + \varepsilon)C(\vec{v}, K - \vec{v})$ for any $\vec{v} \in K$. For $[a, b)$ of cardinality ≥ 2, define $B(\vec{v}, [a, b))$ to be the minimum length of a tree T interconnecting $[a, b) \cup \vec{v}$, with all Steiner points in L, and with the property that either $\vec{v}$ is in K or has valence ≥ 2 in T. Then C and B satisfy the following recursive relations

$$\text{(Pr 1)} \qquad C(\vec{v}, [a, b)) = \min_{\vec{u} \in L}\{B(\vec{u}, [a, b)) + |\vec{u} - \vec{v}|\},$$

$$\text{(Pr 2)} \qquad B(\vec{v}, [a, b)) = \min_{x \in [a, b), x \neq a} \{C(\vec{v}, [a, x)) + C(\vec{v}, [x, b))\}.$$

The algorithm is now to compute all values of C and B by building up a table, using these recursive relations, via dynamic programming. The space usage is $\mathbf{O}(N^4/\varepsilon^2)$ because each table has $\boldsymbol{\theta}(N^2/\varepsilon^2 \times N^2)$ entries. The time usage is $\mathbf{O}(N^6/\varepsilon^4)$ since the time required to compute a value of B is $\mathbf{O}(N)$ and the time required to evaluate a value of C is $\mathbf{O}(N^2/\varepsilon^2)$. Should we actually want the tree $\mathrm{SMT}_L(K)$ rather than merely its length, this is readily accomplished with either a little extra bookkeeping, or else a postprocessing step. (The preprocessing steps of finding CH and computing the smallest square that surrounds the point set, take negligible time in comparison.) □

COROLLARY. *The RMST of N points forming a convex polygon in the plane may be found in time $\mathbf{O}(N^6)$.*

PROOF. Again we rely on Hanan's theorem; instead of using the grid L of $\mathbf{O}(N^2/\varepsilon^2)$ points in the proof of Provan's theorem, we use the N^2-point grid formed by all the horizontal and vertical lines through the N points. □

5.4. How Small Does ε Have To Be? It would be very nice if, in Theorems 13 and 15, and in the algorithm at the beginning of Part II, we could eliminate the dependence of the time bound on ε. (This goal can be accomplished under the assumption of conjectures—see the next section. For a related and simpler, but weaker conjecture, see Problem 3 on p. 331 of this issue.)

It might appear that requiring $1/\varepsilon$ to grow at most polynomially with N is too strong a restriction for most purposes, and one would prefer it if, say, $\log(1/\varepsilon)$ were allowed to grow like some power of N. However, for "most" point sets, in fact, it appears that using $\varepsilon = \mathbf{O}(N^{-2.01})$ is sufficient to obtain the optimum tree, almost always. This claim is made more precise in the following theorem and remarks.

THEOREM 16. *Let N point sites be randomly and independently chosen from the uniform distribution inside the unit d-cube where $d \geq 1$ is fixed, and call the length of their minimum spanning tree L, and of their second-shortest spanning tree L_2. Then*

$$\lim_{N \to \infty} \mathrm{Prob}\{L_2 - L = \mathbf{O}(N^{-3.01})\} = 0.$$

PROOF SKETCH. First, realize that the second shortest spanning tree may be obtained from the MST by removing a single edge, and then rejoining the two subtrees that result with some other edge. Therefore, the difference in their lengths is precisely the difference in lengths between one of the $(N-1)N/2$ line segments joining a pair of sites, and some MST edge.

Draw spherical shells about each site, whose inner radii are the $N-1$ MST edge lengths, and whose thicknesses are each t. Then some intersite distance is $\leq t$ greater (but nevertheless greater) than an MST edge length iff some site lies within some spherical shell. Using this view, it is easy to see that if $t = \mathbf{O}(N^{-3.01})$, then the combined volume of all the shells will be $\mathbf{O}(N^{-1.01})$, hence the probability $\to 0$. □

REMARK 1. In fact, I suspect that one may replace $N^{-3.01}$ by LN^{-C}, where C is any constant with $C > 2$, in this theorem (but not in its proof).

REMARK 2. It is also possible to show (with more difficulty) that the difference in length between the second and second shortest traveling salesman tours is $\Omega(N^{-e})$ with probability $\to 1$, for some fixed positive value e.

REMARK 3. It seems likely that a version of this theorem also holds for the shortest and second best Steiner tree topologies.

I do not have a proof of either of the conjectures of Remarks 1 and 3, however.

In other words, any $(1 + \varepsilon)$-approximation algorithm which works, if $1/\varepsilon$ grows polynomially with N, ought to be suitable for actually finding the *optimum* MST, SMT, or traveling salesman tour, and with a high probability of correctness given random input data. (Of course, one must perform a topology optimization of the output of our $(1 + \varepsilon)$-optimal SMT-finder, to have any hope of finding the exact SMT. But this is a simple postprocessing "beautifying" step.)

5.5. Attack Using a Conjecture about SMT Sensitivity Diagrams. A "geometrical graph" is some graph, composed of line segments, interconnecting N point sites in d-space. The graph is defined by some geometrically motivated rule. (For example, the Delaunay triangulation, minimum spanning tree, optimum traveling salesman tour, and Steiner minimal tree are some interesting geometrical graphs in the plane $\mathbf{R}^2$. A systematic study of geometrical graphs is in Section III.C of [44].)

The "sensitivity diagram" of a geometrical graph on N given sites $\vec{x}_1, \ldots, \vec{x}_N$ in Euclidean d-space is a partition of d-space into a finite number of connected regions R_i, such that if an extra point site $\vec{x}_{N+1}$ were placed somewhere in region R_i, then the geometrical graph of the $N + 1$ sites $\vec{x}_1, \ldots, \vec{x}_{N+1}$ would have a topology invariant under the location of $\vec{x}_{N+1}$. Thus, each *region* in the sensitivity diagram is associated with a *combinatorial structure*, and the sensitivity diagram is completely determined by the locations of the sites $\vec{x}_1, \ldots, \vec{x}_N$ and by the rules defining the geometrical graph under consideration.

It may be shown that the border arcs separating two planar Steiner sensitivity regions are always quartic curves. (This is because, as was mentioned earlier, the square of the length of an optimized FST is a quadratic function of the planar coordinates of the regular sites. The boundary curves are thus defined by the equality of two functions of form $C + \sqrt{\text{quadratic}}$.) Indeed, by using a simple variant of the Hwang–Melzak algorithm, these curves and functions are easily computed if the two associated topologies are given.

Let the maximum complexity (i.e., the number of regions, plus the number of boundary arcs bordering two regions, plus the number of triple points shared by the boundary of ≥ 3 regions) of a Steiner sensitivity diagram on N sites in the plane, be called $f_{\text{SMT}}(N)$.

CONJECTURE 4 (The Polynomial Steiner Diagram Conjecture (PSDC)). *$f_{\mathrm{SMT}}(N)$ is bounded by a polynomial function of N.*

We now present some grounds for believing in this conjecture: we will show that the sensitivity diagrams for the minimum spanning tree (MST), optimum traveling salesman tour (TST), minimum matching (MM), and Delaunay triangulation (DT) (as well as many other geometric graphs, and indeed all these results hold for any fixed dimension d, not just $d = 2$)[7] *do* indeed have sensitivity diagrams of at most polynomially large complexity.

In other words $f_{\mathrm{MST}}(N)$, $f_{\mathrm{TST}}(N)$, $f_{\mathrm{MM}}(N)$, and $f_{\mathrm{DT}}(N)$ *are* polynomially bounded, which fact makes its plausible that $f_{\mathrm{SMT}}(N)$ is too.

THEOREM 17. *The MST, TST, MM, DT have sensitivity diagrams of size at most polynomial(N), in any fixed dimension d.*

PROOF. 1. By Kruskal's algorithm [32], the topology of the MST of N point sites is completely determined by the permutation ordering the $\binom{N}{2}$ distances between the sites. Draw d-spheres of radii d_{ij} about site i, for $i = 1 \cdots N$, where d_{ij} is the distance between sites i, j. These spheres divide d-space into $\mathbf{O}(N^d)$ regions, by Milnor's theorem [38]. (d is fixed.) If the distance ordering is known for this set of N sites, then the distance ordering, upon adding an $(N + 1)$st site, is solely determined by which of these regions the extra site lies in. Therefore, the number of regions in the sensitivity diagram for MST in d-space is $\mathbf{O}(N^d)$. (The fact that the total complexity is also polynomial is now a standard consequence of the "upper bound theorem" [48] or Euler's formula if we are in the plane.)

2. The optimum traveling salesman tour for $N + 1$ sites consists of two edges joining site $N + 1$ to some two sites A and B, glued to the shortest Hamiltonian path with endpoints A, B connecting the first N sites. There are $T \leq \binom{N}{2}$ such topologies in all. The border between any two topologies is an algebraic surface of degree 16 and there are at most $\binom{T}{2}$ such surfaces to consider. Now by Milnor's theorem the total number of connected components in the TST sensitivity diagram is $\mathbf{O}(N^{64d})$.

3. For our purposes, the minimum matching is a set of $\lfloor N/2 \rfloor$ disjoint closed intersite line segments, of minimum possible total length. Adding an extra site X will at worst be exclusive—or the matching with a "alternating" path $XAB \cdots Z$ joining X to some site Z [8]. This path (and its length, except for the distance $\overline{AX}$) is completely determined by A. Therefore, the topology of the new minimum

[7] My Ph.D. thesis [44, Section III.C] contains a large study (including definitions!) of the well-known geometrical graphs mentioned here, as well as many others. Also, [40] contains some discussion of each of these graphs.

matching is completely determined by the old minimum matching and by the region of an "additively weighted Voronoi diagram" [12], [46] of the first N sites which X lies in. (Actually, if N is even, one must also add the contours of zero total length change to this diagram, to allow for the null path.) An upper bound theorem is known [12] for additively weighted Voronoi diagrams, and it follows that the complexity of the MM sensitivity diagram is $\mathbf{O}(N^{d/2+\mathbf{O}(1)})$.[8]

4. Finally, for DT, and a great many other geometrical graphs, one may fall back on Milnor's theorem to prove the result. The general approach is to notice that the geometrical conditions determining a topology are defined by some polynomial (of the d-coordinates of the extra site and the coordinates of the old sites) of bounded degree (in this case, a spherical containment test) and that only a polynomial (in N) number of such conditions can arise. (Namely, in our case, because there are only a polynomial number of d-simplices defined by a set of N point sites!) Applying Milnor's theorem now shows that the complexity of the diagram is $N^{\mathbf{O}(d)}$. The same idea could have been used to prove the three cases above, too, although with some weakening. We will not investigate what specific polynomial bounds may be obtained—considerably stronger results may be given, but they still (probably) fall far short of best possible. □

We will now show that, if the Polynomial Steiner Diagram Conjecture is true, great things will happen.

THEOREM 18. *An algorithm exists, which will find the SMT for any planar set K of N point-sites lying on their own Steiner hull (in particular, any N sites forming a convex N-gon) in time $\mathbf{O}(f_{SMT}(N)^2N^2)$. In particular, if the PSDC is true, the algorithm runs in polynomial time.*

PROOF. We will very much follow Provan's proof of Theorem 15, which was given above—only with some modifications which we now detail. (Again, we will only prove the statement for the convex hull and omit the proof for the Steiner hull.) The difference is that we wish to compute the exact Steiner minimum tree $\mathrm{SMT}(K)$ rather than the approximation $\mathrm{SMT}_L(K)$. Therefore, we replace the finite set L with the infinite set of all points lying on or inside the convex hull (CH). Then, unfortunately, the tables of the functions B and C assume infinite size. To get around this difficulty, we represent $C(\vec{v}, [a, b))$ not as $(N-1)N/2$ discrete tables of function values corresponding to all $\vec{v} \in L$, L finite, but rather as $(N-1)N/2$ piecewise defined functions defined for $\vec{v} \in \mathbf{R}^2$. The pieces arise from the Steiner sensitivity diagram. Specifically, the function $C(\vec{v}, [a, b))$, mapping $\vec{v} \in \mathbf{R}^2$, into $\mathbf{R}$ may be represented as a labeled embedded planar graph. The arcs of the graph are labeled with equations representing the corresponding boundary curve bordering two regions in the Steiner sensitivity diagram; the vertices are labeled by their coordinates; and the faces (regions) R of the graph are associated with the unique tree topology for $\mathrm{SMT}(\vec{v} \cup [a, b))$, $\vec{v} \in R$, corresponding to that sensitivity diagram

[8] In particular, if $d = 2$, then the MM sensitivity diagram is of size $\mathbf{O}(N)$.

region, and the unique formula expressing its length in terms of the coordinates of $\vec{v}$.

B is represented similarly. To determine the diagram $B(\vec{v}, [a, b))$ using the recurrence (Pr 2), overlap the two appropriate C-diagrams $C(\vec{v}, [a, x))$ and $C(\vec{v}, [x, b))$ (as a consequence of Milnor's theorem [38] or Warren's theorem [52], the size of this overlapped diagram is $\mathbf{O}(f_{\mathrm{SMT}}(N)^2)$; all of the points where the boundary curves of one diagram cross curves from the other are readily computed by solving $\mathbf{O}(f_{\mathrm{SMT}}(N)^2)$ equations of bounded degree) and compute the appropriate length function in each region.

Now, computing C via the recurrence relation (Pr 1) requires minimization over an infinite set, but this may be accomplished via a finite number of minimizations of known bounded-degree algebraic functions (each corresponding to a known topology).

Using a variant of the Hwang–Melzak algorithm, one may deduce the length function from the topology in $\mathbf{O}(N)$ time. By brute force, one may find "boundary edges" corresponding to equality of any pair of length-functions, and "triple points" corresponding to triples of putative boundary edges, thus determining a superset of the new combined diagram. Nonexistent edge-segments and triple points may then be detected by comparing function values on them. By taking advantage of region-adjacency information, the entire computation may be organized to take $\mathbf{O}(f_{\mathrm{SMT}}(N)^2)$ time.

The essential thing is that after all of these functions/topologies are found, many of them will turn out to be the same, and many more of them will turn out to be dominated everywhere by others—allowing one to remove redundant boundary edges and nonexistent regions in the new C diagram and once again shrink it down to size $\mathbf{O}(f_{\mathrm{SMT}}(N))$.

The total space usage is then $\mathbf{O}(f_{\mathrm{SMT}}(N)^2)$ (since we may build the B diagrams one at a time and then dispose of them) and the total time usage is $\mathbf{O}(f_{\mathrm{SMT}}(N)^2 N^2)$, since each application of a recurrence takes $\mathbf{O}(f_{\mathrm{SMT}}(N)^2)$ time. (We have been assuming $f_{\mathrm{SMT}}(N) = \mathbf{\Omega}(N)$, which is certainly valid.) □

6. Concluding Remarks. The three foremost open problems raised by this work are:

1. Settle the Polynomial Steiner Diagram Conjecture!
2. Is there a subexponential algorithm for finding the Steiner minimal tree of N sites in the plane?

The results of this paper suggest the existence of an $N^{\mathbf{O}(\sqrt{N})}$-time algorithm for finding the exact Steiner minimum tree of N points in the plane. The author has previously found such algorithms for optimum two-dimensional traveling salesman tour and optimum two-dimensional triangulation [44].

3. Settle the Gilbert–Pollak conjecture in the remaining dimensions ($d = 2$, and $d \geq 10$).

The d-octahedron probably provides a counterexample in $\mathbf{R}^d$ for $d \geq 10$. To prove this would require a determination of the lengths of the SMTs for all the regular

d-octahedra and d-simplices. Possibly some sort of randomized counterexample would also work in high dimensions. In the plane, the conjecture is probably true and one should seek an inductive proof.

The $\mathbf{O}(C^N)$-time algorithm for two-dimensional SMT was previously given in Section III.C.8.2.14–17 of my Ph.D. thesis [44]. The iteration of Section 3 is similar in spirit to the Weiszfeld–Kuhn iteration [7], [33], [34]. In the proof of Theorem 1, the help of Robert J. Holt and of the symbolic algebra system MACSYMA were appreciated.

Appendix 1. Working C Source Code for Finding SMTs in $\mathbf{R}^d$

```
/* Simplified Steiner tree backtrack code   Warren D. Smith Jan 1989
* Program contains about 341 C instructions. See main () for I/O details; for an
* input & output example, see table 1 of the text. */
#include <math.h>
#include <stdio.h>
#define REAL double
#define SQROOT(a) sqrt(a)
#define RANDM(a) frand(a)   /* uniform random deviates in (0, 1) */
#define INITRAND() srand(57731)
#define DoForever while(1)

#define MAXDIM 12   /* max space dimension permitted */
#define NMAX 100   /* max number of sites permitted */
/* Global variables: */
REAL STUB,SCALE,N;
int NUMSITES, DIMENSION;
static int BESTVEC[NMAX],STACK[NMAX*NMAX],adj[NMAX-2][3],edge[2*NMAX][2];
static REAL XX[NMAX*2][MAXDIM],LEN[NMAX],EL[NMAX][3];

/* SMT representation: Steiner point i is adjacent to points adj[i][0..2].
 * (If neighbor is Steiner – NUMSITES added!)
 * There are N – 2 Steiner points and N regular sites.
 * The coordinates of Steiner point i are XX[i+NUMSITES][0..DIMENSION-1], and
 * The coordinates of regular site i are XX[i][0..DIMENSION-1], i=1, 2...
 * Edge i has endpoints edge[i][0]<edge[i][1] (same NUMSITES-add convention).
 ****/

buildtree(k,topvec)
int k, topvec[];
{/* builds tree represented by topvec[1..k]. Initial location of new Steiner pts
  * is a slight random perturbation of the centroid of its neighbors */
  register int i,e,m,j,sn,ea,eb,en;
  /* First: build the tree corresponding to the null vector. */
  N = 3;   m = NUMSITES+1;
  adj[1][0] = 1; adj[1][1] = 2; adj[1][2] = 3;
  edge[1][0] = 1; edge[2][0] = 2; edge[3][0] = 3;
  edge[1][1] = m; edge[2][1] = m; edge[3][1] = m;
  for(i=0; i<DIMENSION; i++)
    XX[m][i] = (XX[1][i]+XX[2][i]+XX[3][i])/3.0 + 0.001*SCALE*RANDM();

  for(i=1; i<=k; i++){ /* Now: do vector element topvec[i] */
    en = i+3; m = i+1; sn = m+NUMSITES;
```

```
    e = topvec[i]; ea = edge[e][0]; eb = edge[e][1];
    adj[m][0] = ea; adj[m][1] = eb; adj[m][2] = en;
    m = ea-NUMSITES;
    if(m>0) for(j=0; j<3; j++) if(adj[m][j]==eb){ adj[m][j] = sn; break; }
    m = eb-NUMSITES;
    if(m>0) for(j=0; j<3; j++) if(adj[m][j]==ea){ adj[m][j] = sn; break; }
    edge[e][1] = sn; e = en+en-4; edge[e][0] = en; edge[e][1] = sn;
    e++; edge[e][0] = eb; edge[e][1] = sn;
    for(j = 0; j<DIMENSION; j++)
      XX[sn][j] = (XX[ea][j]+XX[eb][j]+XX[en][j])/3.0 + 0.001*SCALE*RANDM();
  }
  N = k+3;  /* Tree is now built. Initial coords in general position. */  return;
}

REAL length()
{/* Stores edge lengths of tree T in array EL[1..k1][0..2] and returns total length. */
#define dist(a,b) t=0.0; for(m=0; m<DIMENSION; m++){ r=XX[a][m]-XX[b][m]; t +=r*r; }
t = SQROOT(t);
  register int m,i2,i,j;  int n0,n1,n2,k1;  REAL leng,t,r;
  leng = 0.0; k1 = N-2;
  for(i=1; i<=k1; i++){
    i2 = i+NUMSITES;
    n0 = adj[i][0]; n1 = adj[i][1]; n2 = adj[i][2];
    if(n0<i2){
      dist(n0,i2); leng += t; EL[i][0] = t; n0 -= NUMSITES;
      if(n0>0) for(j=0; j<3; j++) if(adj[n0][j]==i2){ EL[n0][j] = t; break; }
    }
    if(n1<i2){
      dist(n1,i2); leng += t; EL[i][1] = t; n1 -= NUMSITES;
      if(n1>0) for(j=0; j<3; j++) if(adj[n1][j]==i2){ EL[n1][j] = t; break; }
    }
    if(n2<i2){
      dist(n2,i2); leng += t; EL[i][2] = t; n2 -= NUMSITES;
      if(n2>0) for(j=0; j<3; j++) if(adj[n2][j]==i2){ EL[n2][j] = t; break; }
    }
  } /* Have now figured out distance EL[i][0..2] from Steiner pt. i to neighbors. */
  return(leng);
}

optimize(tol)
REAL tol; /* a small positive number */
{/* finds better coordinates XX[NUMSITES+1..NUMSITES+k1][] for the k1 Steiner points
  * of tree T by: doing a relaxation iteration. Assumes edge lengths of old tree
  * have been pre-stored in array EL[][] */
#define prep(a,b,c) if(b>NUMSITES){ val[i]++; B[i][a] = c; }\
else for(m=0; m<DIMENSION; m++) C[i][m] += XX[b][m]*c;
  static REAL B[NMAX][3], C[NMAX][MAXDIM];
  static int eqnstack[NMAX], leafQ[NMAX], val[NMAX];
  register int i,m,j,i2;  int n0,n1,n2,lqp,eqp,k1;  REAL q0,q1,q2,t;

  lqp = eqp = 0; k1 = N-2;
  /* first: compute B array, C array, and valences. Set up leafQ. */
  for(i=k1; i>0; i--){
    n0 = adj[i][0]; n1 = adj[i][1]; n2 = adj[i][2];
    q0 = 1.0/(EL[i][0]+tol); q1 = 1.0/(EL[i][1]+tol); q2 = 1.0/(EL[i][2]+tol);
    /* Have now figured out reciprocal distances q0,q1,q2 from
     * Steiner pt. i to neighbors n0,n1,n2, **/
```

```
  t = q0+q1+q2; q0 /= t; q1 /= t; q2 /= t;
  val[i] = 0; B[i][0] = B[i][1] = B[i][2] = 0.0;
  for(m=0; m<DIMENSION; m++){ C[i][m] = 0.0; }

  prep(0,n0,q0); prep(1,n1,q1); prep(2,n2,q2);
  /* Now: Steiner point i has Steiner valence val[i];
   * coords obey eqns XX[i+NUMSITES][] = sum(j)of B[i][j]*XX[nj][] + C[i][] */

  if(val[i]<=1){ leafQ[lqp] = i; lqp++; } /* put leafs on leafQ */
} /* Have set up equations — now to solve them. */
/* Second: eliminate leaves */
while(lqp>1){
  lqp--; i = leafQ[lqp]; val[i]--; i2 = i+NUMSITES;
  /* now to eliminate leaf i */
  eqnstack[eqp] = i; eqp++; /* push i onto stack */
  for(j=0; j<3; j++) if(B[i][j] != 0.0) break; /* neighbor is #j */
  q0 = B[i][j];
    j = adj[i][j]-NUMSITES; /* neighbor is j */
    val[j]--; if(val[j]==1){ leafQ[lqp] = j; lqp++; } /* new leaf? */
    for(m=0; m<3; m++) if(adj[j][m]==i2) break;
    q1 = B[j][m]; B[j][m] = 0.0;
    t = 1.0-q1*q0; t = 1.0/t;
    for(m=0; m<3; m++) B[j][m] *= t;
    for(m=0; m<DIMENSION; m++){ C[j][m] += q1*C[i][m]; C[j][m] *= t; }
  }
  /* Third: Solve 1-vertex tree! */
  i = leafQ[0]; i2 = i+NUMSITES;
  for(m=0; m<DIMENSION; m++) XX[i2][m] = C[i][m];
  /* Fourth: backsolve */
  while(eqp>0){
    eqp--; i = eqnstack[eqp]; i2 = i+NUMSITES;
    for(j=0; j<3; j++) if(B[i][j] != 0.0) break; /* neighbor is #j */
    q0 = B[i][j];
    j = adj[i][j]; /* neighbor is j */
    for(m=0; m<DIMENSION; m++) XX[i2][m] = C[i][m] + q0*XX[j][m];
  }
  return;
}

REAL error()
{ /* Returns the error figure of tree T with Steiner coords in XX[][].
   * Assumes edge lengths have been pre-stored in array EL[][]. */
  register int i,m,i2,n0,n1,n2;  int k1;  REAL r,s,t,efig,d01,d12,d02;
  k1 = N - 2; efig = 0.0;
  for(i=1; i<=k1; i++){
    i2 = i+NUMSITES;
    n0 = adj[i][0]; n1 = adj[i][1]; n2 = adj[i][2];

    d12 = d01 = d02 = 0.0;
    for(m=0; m<DIMENSION; m++){
      t = XX[i2][m];
      r = XX[n0][m]-t; s = XX[n1][m]-t; t = XX[n2][m]-t;
      d12 += s*t; d01 += r*s; d02 += r*t;
    }
      /* only angles <120 cause error */
```

```
      t = d12 + d12 + EL[i][1]*EL[i][2]; if(t>0.0) efig + = t;
      t = d01 + d01 + EL[i][0]*EL[i][1]; if(t>0.0) efig + = t;
      t = d02 + d02 + EL[i][0]*EL[i][2]; if(t>0.0) efig + = t;
    }
    efig = SQROOT(efig);   return(efig);
}

output_tree()
{
    int i,j;
    printf("topology-describing vector: ");
    for(i=1; i<=NUMSITES-3; i++){ printf("%d ",BESTVEC[i]); }
    printf("\nsteiner point coords\n");
    for(i=NUMSITES+1; i<=2*NUMSITES-2; i++){
      for(j=0; j<DIMENSION; j++) printf("%7g ",XX[i][j]);
      printf("\n");
    }
    printf("edges\n");
    for(i=1; i<2*NUMSITES-3; i++) printf("%d %d\n",edge[i][0],edge[i][1]);
    fflush(stdout);
    return;
}

main(){ /* Inputs NUMSITES, DIMENSION, sites; outputs successive best Steiner
         * trees found. Best tree's topology-vector is stored in BESTVEC. */
    int i,j,x,k,m,nc,ct;   REAL q,r;   int A[NMAX];
    INITRAND();

    /* (1)  Input */
    scanf("%d",&NUMSITES);
    if(NUMSITES<3 || NUMSITES>NMAX){
      fprintf(stderr,"NUMSITES=%d out of range\n",NUMSITES); exit();
    }
    scanf("%d",&DIMENSION);
    if (DIMENSION<2 || DIMENSION>MAXDIM){
      fprintf(stderr,"DIMENSION=%d out of range\n",DIMENSION); exit();
    }
    printf("%d %d\n",NUMSITES,DIMENSION);
    for(i=1; i<=NUMSITES; i++) for(j=0; j<DIMENSION; j++) scanf("%f",&(XX[i][j]));
    SCALE = 0.0;
    for(i=1; j<=NUMSITES; i++){
      for(j=0; j<DIMENSION + j++){
        q = XX[i][j] - XX[1][j]; if(q<0.0) q = -q; if(q<SCALE) SCALE = q;
        printf("%g ",XX[i][j]);
      }
      printf("\n");
    }
    fflush(stdout);

    if(NUMSITES==3){ /* Deal with special case of 3 sites */
      buildtree(0,A);
      q = length(); r = error();
      do{ optimize(0.0001*r/NUMSITES); }while(r>q*0.0001);
      printf("\nnew record length %g\n",q);
      output_tree();
```

```
    printf("done\n"); fflush(stdout); exit();
  }

  /* (2) Preprocessing and initialization */
  /* Optionally, sort sites in some nice order here. */
  /* STUB = any upper bound on the length of the SMT. */
  STUB = HUGE; k=1; m=0;   ct=0; /* ct counts backtrack iters. Unused at present */

  DoForever{ /* 3: candidate leaf generation and backtracking */
    nc=0; ct++;
/* if(ct%100000==0){ printf("still running!\n"); fflush(stdout); } */
    for(x=2*k+1; x>0 && k<=NUMSITES-3; x--){
      A[k] = x;
      buildtree(k,A); /* Build the tree represented by the topol. vector A[1..k] */
ITER: /* ...and optimize it until either obviously bad or small error figure happens */
      q = length();
      r = error();
      if(q-r<STUB){
        if(r>0.005*q){ optimize(0.0001*r/NUMSITES); goto ITER; }
        if(k>=NUMSITES-3){
          do{ optimize(0.0001*r/NUMSITES); q = length(); r = error();
          }while(r>q*0.0001);
          if(q<STUB){
            printf("\nnew record length %20.20g\n",q);
            for(i=1; i<=k; i++) BESTVEC[i]=A[i];
            if(q<STUB*0.99999) output_tree();
            STUB=q;
          }
        }
        else{
          i = nc; nc++;
          while(i>0 && LEN[i]<q){
            STACK[m+i+1]=STACK[m+i]; LEN[i+1]=LEN[i]; i--;
          }
          i = i+1; STACK[m+i]=x; LEN[i] = q;
        }
      }
    }
    m = m+nc;

    while(nc<=0){
      k--;
      if(k<=0){ printf("done\n"); fflush(stdout); exit(); }
      nc = STACK[m]; m--;
    }

    A[k]=STACK[m]; STACK[m]=nc-1;
    if(k<NUMSITES-3) k++; else m--;
  }
} /* End of Steiner tree program. */
```

Appendix 2

Table 3. SMTs for regular d-simplices S_d, $d = 3 \cdots 8$.

S_3. Length 3.44949. Steiner point coordinates:

0.295875854768068	0.295875854768068	0.204124145231932	0.204124145231932
0.204124145231932	0.204124145231932	0.295875854768068	0.295875854768068

edges 1–5 2–5 3–6 4–6 5–6

S_4. Length 4.43355. Steiner point coordinates:

0.27758	0.27758	0.131133	0.131133	0.182574
0.131133	0.131133	0.27758	0.27758	0.182574
0.18545	0.18545	0.18545	0.18545	0.258199

edges 1–6 2–6 3–7 4–7 6–8 5–8 7–8

S_5. Length 5.40629. Steiner point coordinates:

0.264298	0.264298	0.117851	0.117851	0.117851	0.117851
0.166667	0.166667	0.166667	0.166667	0.166667	0.166667
0.117851	0.117851	0.264298	0.117851	0.264298	0.117851
0.117851	0.117851	0.117851	0.264298	0.117851	0.264298

edges 1–7 2–7 3–9 4–10 7–8 5–9 8–9 6–10 8–10

S_6. Length 6.37607. Steiner point coordinates:

0.110376	0.256822	0.256822	0.0792688	0.110376	0.0792688	0.107066
0.156095	0.156095	0.156095	0.112103	0.156095	0.112103	0.151415
0.256822	0.110376	0.110376	0.0792688	0.256822	0.0792688	0.107066
0.0792688	0.0792688	0.0792688	0.260132	0.0792688	0.260132	0.162661
0.110005	0.110005	0.110005	0.167125	0.110005	0.167125	0.225732

edges 1–10 2–8 3–8 4–11 8–9 5–10 9–10 6–11 9–12 7–12 11–12

S_7. Length 7.33814. Steiner point coordinates:

0.104608	0.251055	0.251055	0.0721688	0.104608	0.0721688	0.0721688	0.0721688
0.147938	0.147938	0.147938	0.102062	0.147938	0.102062	0.102062	0.102062
0.251055	0.104608	0.104608	0.0721688	0.251055	0.0721688	0.0721688	0.0721688
0.102062	0.102062	0.102062	0.147938	0.102062	0.147938	0.147938	0.147938
0.0721688	0.0721688	0.0721688	0.251055	0.0721688	0.104608	0.251055	0.104608
0.0721688	0.0721688	0.0721688	0.104608	0.0721688	0.251055	0.104608	0.251055

edges 1–11 2–9 3–9 4–13 9–10 5–11 10–11 6–14 10–12 7–13 12–13 8–14 12–14

S_8. Length 8.30163. Steiner point coordinates:

0.096068	0.142892	0.142892	0.142892	0.0708659	0.142892	0.096068	0.0708659	0.09
0.0679303	0.10104	0.247486	0.247486	0.0501097	0.10104	0.0679303	0.0501097	0.06
0.13727	0.0969541	0.0969541	0.0969541	0.10126	0.0969541	0.13727	0.10126	0.13
0.0679303	0.247486	0.10104	0.10104	0.0501097	0.247486	0.0679303	0.0501097	0.06
0.245856	0.0679303	0.0679303	0.0679303	0.0709468	0.0679303	0.245856	0.0709468	0.09
0.0709468	0.0501097	0.0501097	0.0501097	0.252041	0.0501097	0.0709468	0.252041	0.15
0.0977086	0.0690116	0.0690116	0.0690116	0.158509	0.0690116	0.0977086	0.158509	0.21

edges 1–14 2–13 3–11 4–11 10–11 5–15 10–12 6–13 10–13 7–14 12–14 8–15 12–16 9–16 15–16

Table 4. SMTs for regular d-octahedra G_d, $d = 3 \cdots 6$.

G_3. Length 5.73603. Steiner point coordinates:

−0.0451759	0.113105	0.238327
−0.203016	0.262427	0.151357
0.238327	−0.113105	−0.0451759
0.151357	−0.262427	−0.203016

edges 1–9 2–8 3–7 4–8 7–8 5–10 7–9 6–10 9–10

G_4. Length 7.60993. Steiner point coordinates:

−0.126662	0.0849164	0.242564	0.145241
−.0030127	−0.0275614	0.121316	0.233117
−.0261223	0.22438	0.158258	0.0947611
0.121316	−0.233117	−0.030127	0.0275614
0.242564	−0.145241	−0.126662	−0.0849164
0.158258	−0.0947611	−0.261223	−0.22438

edges 1–13 2–11 3–9 4–10 9–10 5–11 9–11 6–12 10–12 7–14 12–13 8–14 13–14

G_5. Length 9.49174. Steiner point coordinates:

−0.139548	0.113213	0.232887	0.146942	0.0909957
−0.0567576	0.0215602	0.126549	0.227157	0.140669
0.0125803	−0.0649318	0.0501403	0.125071	0.223471
−0.258809	0.235279	0.157674	0.0994862	0.061608
−0.125071	−0.223471	−0.0501404	0.0125802	0.0649318
0.227157	−0.14067	−0.126549	−0.0567576	−0.0215602
0.146942	−0.0909957	−0.232887	−0.139548	−0.113213
0.0994862	−0.061608	−0.157674	−0.258809	−0.235278

edges 1–16 2–14 3–11 4–12 11–12 5–13 12–13 6–14 11–14
7–15 13–15 8–17 15–16 9–18 16–17 10–18 17–18

G_6. Length 11.3871. Steiner point coordinates:

0.248866	0.229792	0.0871722	0.0972762	0.0550536	0.102881
0.118823	0.0973494	0.131062	0.146253	0.0827721	0.15468
0.0423294	0.0113301	0.215559	0.07013	0.136137	0.0819851
0.0897705	0.0735473	0.0990168	0.23203	0.0625341	0.239826
−0.0206923	−0.0674238	0.124823	0.0135015	0.216114	0.0311705
−0.119159	−.0203756	0.0324699	−.00647183	0.0821438	−.00329286
−0.204569	−0.13252	−0.0192047	−0.126664	0.0140906	−0.0797282
−0.13648	−0.088412	−0.0731208	−0.210995	−0.04943	−0.140405
−0.0637951	−0.0413266	−0.25709	−0.0986258	−0.240554	−0.157838
−0.0930497	−0.0602778	−0.142854	−0.143853	−0.124424	−0.230218

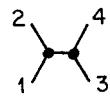
edges 1–13 2–13 3–15 4–16 13–14 5–17 14–15 6–16 14–16 7–19
15–17 8–18 17–18 9–21 18–19 10–20 19–20 11–21 20–22 12–22 21–22

Table 5. SMTs for 2-cube C_2 and 3-cube C_3.

C_2, length = 2.73205, topology = (3)

Steiner point coordinates:

0.5	0.711325
0.5	0.288675



C_3, length = 6.19615, topology = (2, 1, 2

Steiner point coordinates:

0.5	0.5	0.288675
0.5	0.5	0.711325
0.5	0.25	0.144338
0.5	0.25	0.855662
0.5	0.75	0.144337
0.5	0.75	0.855663

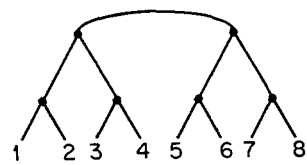


Table 6. SMT for 10-vertex Archimedean polytope in 4-space.

Length 49.9952. Steiner point coordinates:

−1.75214	−0.561441	0.939928	1.27521	0.108343
−1.11658	1.69864	0.181338	−0.28443	−0.478968
1.51831	1.14318	−1.00237	−1.13314	−0.525978
−1.43339	1.28849	0.0349657	0.673069	−0.563131
2.28574	−0.484812	−0.791394	−0.869277	−0.140259
−0.490872	1.75203	−0.528171	−0.818327	0.0853377
2.53443	−1.01237	−0.332783	−0.401492	−0.787786
−1.84935	−1.12563	0.717619	1.01493	1.24243

Table 7. SMT for 16-vertex "half-cube" polytope in 5-space.

H_5. Length 15.9585. Steiner point coordinates:

0.333306	0.417593	0.443431	0.328399	0.709654
0.302163	0.38323	0.42133	0.300505	0.291792
0.249508	0.564155	0.314099	0.432189	0.84021
0.344867	0.437976	0.540929	0.692895	0.0896296
0.356371	0.454048	0.5055	0.355654	0.188055
0.254906	0.55489	0.299659	0.663279	0.905242
0.425638	0.306745	0.571131	0.251931	0.831677
0.424494	0.538286	0.446142	0.804188	0.0571485
0.432689	0.554391	0.591301	0.219199	0.115903
0.631288	0.531079	0.563611	0.142464	0.075329
0.617434	0.523712	0.462459	0.869834	0.0379893
0.664437	0.294873	0.559441	0.257292	0.90166
0.340302	0.719824	0.379317	0.78805	0.940354
0.787937	0.374896	0.721583	0.342089	0.937853

References

[1] A. V. Aho, M. R. Garey, and F. K. Hwang: Rectilinear Steiner trees: Efficient special case algorithms, *Networks*, **7** (1977), 37–58.

[2] M. Ajtai, V. Chvatal, M. M. Newborn, and E. Szemeredi: Crossing free subgraphs, *Ann. Discrete Math.*, **12** (1982), 9–12.

[3] S. K. Chang: The generation of minimal trees with a Steiner topology, *J. Assoc. Comput. Math.*, **19** (1972), 699–711.

[4] F. R. K. Chung and E. N. Gilbert: Steiner trees for the regular simplex, *Bull. Inst. Math. Acad. Sinica*, **4** (1976), 313–325.

[5] F. R. K. Chung and R. L. Graham: A new bound for Euclidean Steiner trees, *Ann. N.Y. Acad. Sci.*, **440** (1985), 328–346.

[6] E. J. Cockayne: On Fermat's problem on the surface of a sphere, *Math. Mag.*, **45** (1972), 216–219.

[7] E. J. Cockayne and D. E. Hewgill: Exact computation of Steiner minimal trees in the plane, *Inform. Process. Lett.*, **22** (1986), 151–156.

[8] U. Derigs: A shortest augmenting path method for solving minimum perfect matching problems, *Networks*, **11** (1981), 379–390.

[9] D. Z. Du: On Steiner ratio conjectures, manuscript, Inst. Appl. Math. Academia Sinica, Beijing, China, 1989.

[10] D. Z. Du: The Steiner ratio conjecture is true for six points, to be published.

[11] D. Z. Du, F. K. Hwang, and J. F. Weng: Steiner minimal trees for regular polygons, *Discrete Comput. Geom.*, **2**, 1 (1987), 65–87.

[12] H. Edelsbrunner: *Algorithms in Combinatorial Geometry*, EATCS Monographs in Theoretical Computer Science, Springer-Verlag, Berlin, 1987.

[13] J. H. Friedman, J. L. Bentley, and R. A. Finkel: An algorithm for performing best matches in logarithmic expected time, *ACMTOMS*, **3**, 3 (1977), 209–226.

[14] M. R. Garey and D. S. Johnson: The complexity of computing Steiner minimum trees, *SIAM J. Algebraic Discrete Methods*, **32** (1977), 835–859.

[15] M. R. Garey and D. S. Johnson: The rectilinear Steiner minimum tree problem is NP-complete *SIAM J. Algebraic Discrete Methods*, **32** (1977), 826–834.

[16] E. Gekeler: On the solution of systems of equations by the epsilon algorithm of Wynn, *Math. Comp.*, **26**, 118 (1972), 427–436.

[17] G. Georgakopoulos and C. H. Papadimitriou: The 1-Steiner tree problem, *J. Algorithms*, **8** (1987), 122–130.

[18] E. N. Gilbert and H. O. Pollak: Steiner minimal trees, *SIAM J. Appl. Math.*, **16** (1968), 1–29.

[19] R. L. Graham and F. K. Hwang: Remarks on Steiner minimal trees I, *Bull. Inst. Math. Acad. Sinica*, **4** (1976), 177–182.

[20] M. Hanan: On Steiner's problem with rectilinear distance, *SIAM J. Appl. Math.*, **14** (1966), 255–265.

[21] K. Helbig-Hansen and J. Krarup: Improvements of the Held–Karp algorithm for the symmetric TSP, *Math. Programming*, **7** (1974), 87–96.

[22] M. Held and R. M. Karp: The traveling salesman problem and minimum spanning trees, *Oper. Res.*, **18** (1970), 1138–1162.

[23] M. Held and R. M. Karp: The traveling salesman problem and minimum spanning trees, part II, *Math. Programming*, **1** (1971), 6–25.

[24] J. E. Hopcroft and R. E. Tarjan: Efficient planarity testing, *J. Assoc. Comput. Math.*, **21** (1974), 549–558.

[25] F. K. Hwang: A linear time algorithm for full Steiner trees, *Oper. Res. Lett.*, **5** (1986), 235–237.

[26] F. K. Hwang and D. S. Richards: Steiner tree problems, Bell Laboratories, Murray Hill, NJ, Technical Memorandum 1989. To be published in *Networks*.

[27] F. K. Hwang, G. D. Song, G. Y. Ting, and D. Z. Du: A decomposition theorem on Euclidean Steiner minimal trees, *Discrete Comput. Geom.*, **3**, 4 (1988), 367–392.

[28] F. K. Hwang and J. F. Weng: Hexagonal coordinate systems and Steiner minimal trees, *Discrete Math.*, **62** (1986), 49–57.

[29] F. K. Hwang and J. F. Weng: The shortest network with a given topology, Bell Laboratories, Murray Hill, NJ, Technical Memorandum 1988.

[30] A. N. C. Kang and D. A. Ault: Some properties of the centroid of a free tree, *Inform. Process. Lett.*, **4**, 1 (1975), 18–20.

[31] B. Kernighan and D. Ritchie: *The C Programming Language*, Prentice-Hall, Englewood Cliffs, NJ, 1978.

[32] J. R. Kruskal Jr.: On the shortest spanning subtree of a graph and the traveling salesman problem, *Proc. Amer. Math. Soc.*, **7**, 1 (1956), 48–50.

[33] H. W. Kuhn: On a pair of dual nonlinear programs, in: *Methods of Nonlinear Programming*, pp. 38–54, Ed. J. Abadie, North-Holland, Amsterdam, 1967.

[34] H. W. Kuhn: A note on Fermat's problem, *Math. Programming*, **4** (1973), 98–107.

[35] E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan, and D. B. Schmoys (eds.) *The TST, A Guided Tour of Combinatoral Optimization*, Wiley-Interscience, New York, 1985.

[36] Z. A. Melzak: On the problem of Steiner, *Canad. Math. Bull.*, **4** (1961), 143–148.

[37] G. L. Miller: Finding small simple cycle separators for 2-connected planar graphs, *J. Comput. System Sci.*, **32** (1986), 265–179.

[38] J. Milnor: On the Betti numbers of real algebraic varieties, *Amer. Math. Soc.*, **15** (1964), 275–280.

[39] C. Monma, M. Paterson, S. Suri, and F. Yao: Computing Euclidean maximum spanning trees, ACM Computational Geometry Conference 1988, pp. 241–251.

[40] F. P. Preparata and M. I. Shamos: *Computational Geometry: An Introduction*, Springer-Verlag, New York, 1985.

[41] R. C. Prim: Shortest Connection Networks and Some Generalizations, *Bell System Tech. J.*, **36** (1957), 1389–1401.

[42] J. S. Provan: Convexity and the Steiner tree problem, *Networks*, **18** (1988), 55–72.

[43] P. W. Shor and W. D. Smith: Steiner hulls and θ-hulls, manuscript, 1989.

[44] W. D. Smith: Studies in Discrete and Computational Geometry, Ph.D. Thesis, Program in Applied and Computational Mathematics, Princeton University, September 1988.

[45] D. Smith: Finding the optimum traveling salesman tour for N sites in the Euclidean plane in subexponential time and polynomial space, *SIAM J. Comput.*, submitted.

[46] J. M. Smith, D. T. Lee, and J. S. Liebman: A $O(N \lg N)$ algorithm for Steiner minimal tree problems in the Euclidean metric, *Networks*, **11** (1981), 23–29.

[47] J. Soukup: Minimum Steiner trees, roots of a polynomial, and other magic, *ACM SIGMAP Newsletter*, **22** (1977), 37–51.

[48] R. P. Stanley: The number of faces of simplicial polytopes and spheres, in: Discrete Geometry and Convexity, *Proc. N. Y. Acad. Sci.*, (1986).

[49] R. E. Tarjan: Data structures and network algorithms, CBMS-NSF Regional Conference Series in Applied Mathematics, vol. 44, Society of Industrial and Applied Mathematics, Philadelphia, PA, 1983.

[50] J. V. Uspensky: *Theory of Equations*, McGraw-Hill, New York, 1948.

[51] B. L. van der Waerden: *Algebra*, Ungar, New York, 1970.

[52] H. E. Warren: Lower Bounds for approximation by nonlinear manifolds, *Trans. Amer. Math. Soc.*, **133** (1968), 167–178.

[53] P. Winter: An algorithm for the Steiner problem in the Euclidean Plane, *Networks*, **15** (1985), 323–345.